Notes de cours INFO702, M1 Informatique
Programmation générique et C++

Jacques-Olivier Lachaud
LAMA, Université de Savoie
jacquesolivierlachaud.github.io
(suivre Teaching/INFO702)

27 octobre 2026

Introduction

Ce document présente les principaux mécanismes de la programmation générique en C++. Il s'adresse à des étudiants ayant acquis les bases d'une licence en informatique et connaissant déjà la programmation impérative et les principes de la programmation objet. Il suppose une connaissance du langage C, notamment de sa syntaxe, des pointeurs, de l'allocation mémoire, de l'organisation en fichiers source et en-tête et des étapes de compilation et d'édition des liens. Les notes du cours INFO505 – Programmation C, disponibles sur le même site, peuvent servir de rappel. Une familiarité avec les classes, les méthodes et le polymorphisme sera également utile. Certaines notions seront comparées à celles de Java, mais la connaissance de Java n'est pas nécessaire.

Le cours introduit les éléments du langage utiles à la programmation générique, puis les patrons de fonctions et de classes, les conteneurs, les itérateurs et les algorithmes de la bibliothèque standard. Il aborde ensuite des techniques plus avancées : concepts, contraintes, traits, sélection d'implémentations à la compilation et métaprogrammation. Les expressions lambda, les exceptions et quelques autres outils du langage sont présentés dans la dernière partie. Le document ne cherche pas à couvrir toute la syntaxe ni l'ensemble de la bibliothèque standard.

Versions du langage. Le C++ est normalisé par l'ISO et évolue par révisions successives : C++98 et C++03, puis C++11, C++14, C++17, C++20 et C++23. Le nom « C++0x » désignait le projet devenu C++11 ; il ne s'agit pas d'un standard distinct. C++11 a notamment introduit les expressions lambda, les références de rvalue, une nouvelle déduction avec `auto`, les pointeurs intelligents et une bibliothèque standard pour les threads. C++14 et C++17 ont apporté de nouvelles fonctionnalités, par exemple les lambdas génériques, `if constexpr`, les liaisons structurées, `std::optional`, `std::filesystem` et les politiques d'exécution de certains algorithmes standard.

Le cours prend C++11 comme base pour les exemples et utilise largement C++20, notamment pour les concepts, les ranges, les concepts d'itérateurs et `std::format`. Quelques apports de C++23 sont aussi signalés. Les exemples indiquent leur version minimale lorsqu'elle est postérieure à C++11 ; les fragments signalés comme illustratifs ne constituent pas nécessairement des programmes complets. Certaines expressions servent à montrer une erreur et sont alors commentées ou identifiées comme invalides.

Pour compiler un programme C++20 avec GCC, on peut utiliser :

```
g++ -std=c++20 -Wall -Wextra -Wpedantic programme.cpp -o programme
```

Le choix de l'option du langage ne garantit pas que toutes les fonctionnalités correspondantes sont implémentées : le compilateur et sa bibliothèque standard doivent les prendre en charge. En particulier, certains exemples utilisant `std::format` ou des fonctionnalités C++23 nécessitent une bibliothèque suffisamment récente.

Ressources. Pour approfondir le langage et la programmation générique, on pourra consulter les ouvrages de Bjarne Stroustrup, Scott Meyers et Herb Sutter, ou ceux de Deitel et Deitel pour une introduction plus progressive. Alexander Stepanov et ses collaborateurs ont joué un rôle important dans le développement de la programmation générique et de la STL.

Le site `https://en.cppreference.com/` fournit une documentation détaillée du langage et de la bibliothèque standard ; `https://cplusplus.com/` propose également une référence et des tutoriels. Le projet public du standard est consultable sur `https://eel.is/c++draft/` ; il évolue et ne correspond pas toujours à une norme publiée. Les *C++ Core Guidelines*, notamment dirigées par Bjarne Stroustrup et Herb Sutter, proposent des conseils de conception et de programmation : `https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines`.

Les bibliothèques Boost (`https://www.boost.org/`) complètent la bibliothèque standard ; certaines ont inspiré des fonctionnalités ensuite intégrées au standard. La bibliothèque `boost::program_options` aide à gérer les arguments d'un programme. `boost::compute` permet d'écrire des calculs destinés à des dispositifs OpenCL.

Plan

1. Éléments du langage : variables, pointeurs, références, fonctions et classes.

2. Programmation orientée objet : encapsulation, héritage et polymorphisme.
3. Gestion de la mémoire : allocation, durée de vie et pointeurs intelligents.
4. Patrons de fonctions et de classes : déduction, concepts et spécialisations.
5. Bibliothèque standard : conteneurs, itérateurs, algorithmes et ranges.
6. Programmation générique avancée : contraintes, traits, SFINAE et métaprogrammation.
7. Expressions lambda, exceptions et autres outils du langage.

1 Quelques éléments du langage : syntaxe, variables, pointeurs, références, fonctions, classes

Le C++ est un langage généralement compilé et à typage statique : le compilateur vérifie les types avant l'exécution et produit du code pour une architecture donnée. Cette vérification ne garantit pas l'absence d'erreurs à l'exécution : un accès hors limites ou un pointeur invalide restent possibles. Python effectue davantage de vérifications de types pendant l'exécution.

La version minimale du standard est indiquée seulement lorsqu'elle est postérieure à C++11. Les fragments illustratifs ne constituent pas toujours des programmes complets ; les lignes signalées comme invalides servent à expliquer les diagnostics du compilateur. On recommande C++20 pour suivre ce cours, avec une bibliothèque standard qui implémente les fonctionnalités utilisées.

1.1 Le programme "Hello world !"

En C++, il ressemble à ceci (fichier appelé `hello.cpp`) :

```
#include <iostream>
int main( int argc, char** argv )
{
  std::cout << "Hello World !" << std::endl;
  return 0;
}
```

Cela ressemble à du C, mais la gestion de l'affichage sur la sortie standard est plus agréable qu'en C. Ici `std::cout` désigne une variable globale `cout` définie dans l'espace de noms `std`, c'est-à-dire la bibliothèque standard C++ (la STL en constitue une partie). Le type de cette variable est un *flux* de sortie (classe `std::ostream`). On remarque tout de suite une spécificité du C++ par rapport au C : l'opérateur `<<`, qui désigne habituellement le décalage à gauche des bits d'un entier, désigne ici un moyen pour envoyer la chaîne de caractères `"Hello World !"` sur le flux `cout`.

Autrement dit, en C++, les opérateurs usuels peuvent être redéfinis pour chaque type déclaré : c'est la *surcharge* des opérateurs.

NB : En C, on ne peut même pas surcharger une fonction (un nom de fonction = une fonction), alors encore moins surcharger des opérateurs.

Questions : en Java et en Python, peut-on surcharger les opérateurs ? Peut-on définir plusieurs fonctions de même nom avec des paramètres différents ?

On retrouve sinon le *préprocesseur* du C, avec les directives `#include`. Ici, la directive inclut le fichier en-tête `iostream`. Celui-ci contient toutes les déclarations propres à la gestion des flux pour les entrées/sorties standards. Pour les flux liés à des fichiers, on inclut `fstream`.

On écrira pour compiler ce fichier sur votre shell :

```
# Compilation
(you@machine) g++ -c hello.cpp
```

Si la compilation réussit, elle produit un fichier objet `hello.o`. Celui-ci n'est pas encore un exécutable : l'édition des liens résout les références aux fonctions et objets des bibliothèques utilisées. On écrit :

```
# Compilation
(you@machine) g++ hello.o -o hello
```

Cela fabrique un exécutable `hello`, que l'on peut exécuter, ce qui donne :

```
# Compilation
(you@machine) ./hello
Hello World !
```

Ici, la commande `g++` (sans le `-c`) fait l'édition des liens avec toute la bibliothèque système C++, qui contient entre autres les fonctions d'entrées/sorties sur la console.

Choisissez explicitement le standard et activez les avertissements :

```
# C++20 ; remplacer c++20 par c++11, c++17, etc. si necessaire.
g++ -std=c++20 -Wall -Wextra -Wpedantic hello.cpp -o hello
```

L'option `-std=c++20` active les règles du langage correspondantes ; elle ne complète pas une bibliothèque standard ancienne. Pour le premier programme, `int main()` suffit si les arguments ne sont pas utilisés. `std::endl` ajoute un saut de ligne et force le vidage du flux ; `'\n'` suffit généralement.

1.2 Les variables ; portée d'une variable

Une variable possède notamment un nom et un type. La *portée* indique où son nom est utilisable ; sa *durée de vie* indique quand l'objet existe. Ces deux notions sont distinctes. Préférez des variables locales et limitez leur portée au bloc qui en a besoin.

1.2.1 Définition d'une variable.

Une variable est *définie* en écrivant un *type* suivi d'un *identifiant*, qui sera le nom de la variable. La définition d'une variable entraîne la création de l'objet correspondant. La taille mémoire réservée dépend du type voulu. Par exemple, un entier nécessite un nombre d'octets dépendant de son type et de l'implémentation, tandis qu'un long texte peut prendre quelques milliers d'octets.

```
void f() {
  int i; // Définition d'une variable de type ``int'', non initialisée
  int j = 3; // Définition d'une variable de type ``int'', initialisée à 3
}
```

1.2.2 Déclaration d'une variable.

Déclarer une variable (globale) est une indication pour le compilateur qu'une telle variable (globale) existe ailleurs dans le programme. C'est souvent seulement à l'*édition des liens* que cette existence supposée est vérifiée. Il faut utiliser le mot-clé `extern`.

```
// Fichier toto.hpp
extern const double mon_pi; // déclaration.
...
```

```
// Fichier toto.cpp
#include "toto.hpp" // reprend la declaration extern
const double mon_pi = 3.14159; // définition.
```

1.2.3 Variables globales.

Une variable *définie* en dehors de toute fonction a une portée dite *globale*. Elle est accessible directement à partir de toute fonction définie *après* l'endroit où cette variable était définie.

```
void f()
{
  ma_variable_globale = 7; // Erreur, non encore déclarée
}
// Définition d'une variable globale de nom ``ma_variable_globale''
int ma_variable_globale;
...
void g()
{
  f();
  std::cout << "elle vaut: "
            << ma_variable_globale    // valide
            << std::endl;
}
```

Si vous voulez vous servir d'une variable globale, soit avant dans le même module, soit dans un autre module, on utilise le mot-clé **extern** pour *déclarer* la variable. Cela donnerait pour l'exemple ci-dessus :

```
// Déclaration d'une variable globale de nom ''ma_variable_globale''
extern int ma_variable_globale;
...
void f()
{
  ma_variable_globale = 7; // Tout va bien.
}
...
// Définition d'une variable globale de nom ''ma_variable_globale''
int ma_variable_globale;
...
```

1.2.4 Variables locales.

Un paramètre et une variable définie dans un bloc ont une portée locale. Une variable locale ordinaire est construite lorsque l'exécution atteint sa définition et détruite à la sortie de son bloc. Une variable locale `static`, en revanche, persiste entre les appels.

```
int mult2( int i )
{ // i est une variable locale à la fonction mult2.
  return 2*i;
}
int carre( int i )
{ /* i est une variable locale à la fonction carre. Elle porte le
  même nom que la variable i de la fonction mult2, mais n'a
  strictement rien à voir. */
  return i*i;
}
// Cette fonction retourne (j+1)*(j+1) + 2*j + 1.
int f( int j )
{
  int val = carre( j+1 );
  val += mult2( j ) + 1;
  return val;
}
```

La variable `j` de `f` reste vivante pendant les appels à `carre` et `mult2`, mais son nom n'est pas visible dans ces fonctions. On peut leur donner accès à l'objet par un pointeur ou une référence.

1.2.5 Initialiser les variables ; déduire leur type.

Évitez les variables scalaires non initialisées. Les accolades permettent une initialisation uniforme et interdisent certaines conversions avec perte (*narrowing*). Le mot-clé `auto` (depuis c++11) déduit le type à partir de l'initialiseur : le typage reste statique.

```
// fragment dans une fonction
int n{};           // zero
int m{3};
double x{2.5};
auto copie = x;    // double
const auto limite = 100; // const int
// int erreur{2.5}; // invalide : conversion avec perte
```

Pour les conteneurs, accolades et parenthèses peuvent avoir des sens différents : `std::vector<int> a(3, 7)` contient trois fois 7, tandis que `std::vector<int> b{3, 7}` contient les deux entiers 3 et 7 (en-tête `<vector>`).

1.3 Pointeurs ; tableaux

Le C++, comme le C, permet d'utiliser le mécanisme des *pointeurs*, notamment pour accéder à des variables ou des zones mémoire en dehors des variables locales propres à la fonction courante. Modulo le mot-clé `const`, utilisé pour contraindre le champ d'utilisation d'un pointeur, la gestion des pointeurs en C++ est identique à celle du langage C.

```
// [Utilisation de pointeurs (méthode C... ou C++)]
// fonction qui échange les valeurs pointées par pi et pj
void swap(int* pi, int* pj)
{ // pi pointe sur la case mémoire stockant i
```

```
  // pj pointe sur la case mémoire stockant j
  int tmp = *pi;
  *pi = *pj;
  *pj = tmp;
}

int main()
{
  int x = 1;
  int y = 2;
  swap( &x, &y ); // après, x = 2, y = 1
}
```

On peut contraindre les pointeurs à pointer sur des zones non-modifiables (type `const T*`) ou à ne pouvoir être déplacé (type `T* const`) ou les deux à la fois (type `const T* const`).

Sur l'exemple ci-dessus on aurait pu écrire :

```
void swap(int* const pi, int* const pj) // le reste inchangé
```

Un *tableau* contient un nombre fixé d'éléments contigus du même type. Ses indices vont de zéro à sa taille moins un. Dans de nombreuses expressions, son nom est converti en pointeur vers son premier élément ; le tableau lui-même n'est pourtant pas un pointeur.

```
// fragment
char c[10];                 // elements non initialises si variable locale
char d[] = "Bonjour";       // 8 elements, dont le zero final
const char* e = "Bonjour"; // pointeur vers un litteral non modifiable
int t[] = {5, 4, 3, 2, 1};
int* p = t;                 // conversion tableau -> pointeur
int (*pt)[5] = &t;          // pointeur vers le tableau entier
```

`t` et `&t` n'ont donc pas le même type. Le tableau ne peut être affecté globalement, alors qu'un pointeur peut changer de valeur. L'écriture `char* e = "Bonjour"` est interdite depuis C++11 (et était dépréciée auparavant).

Pour un tableau de taille fixe, `std::array` conserve explicitement la taille dans le type et offre une interface de conteneur ; pour une taille variable, on utilisera `std::vector`.

```
// fragment ; #include <array>
std::array<int, 3> a{{1, 2, 3}};
a[0] = 7;           // acces sans verification de l'indice
a.at(1) = 8;        // leve std::out_of_range si l'indice est invalide
auto taille = a.size();
int* p = a.data();
```

Depuis C++11, `nullptr` désigne un pointeur nul, de préférence à 0 ou `NULL`. Ne déréférencez un pointeur que s'il désigne un objet vivant.

1.4 Références

Le C++ dispose de plus du mécanisme de *référence*, qui simplifie dans beaucoup de cas l'écriture du code C++ par rapport au C, tout en ayant la même efficacité. Une référence est souvent utilisée lors d'un passage en **entrée/sortie** ou en **sortie** d'un paramètre. Le paramètre formel devient un *alias* de l'argument d'appel dès sa construction. Ensuite, tout se passe comme si le paramètre formel **est** l'argument d'appel. Les deux noms désignent le même objet. Les références se déclarent avec le symbole `&`.

```
// [Utilisation de références (méthode C++)]
// fonction qui échange les valeurs de i et j
void swap(int& i, int& j)
{ // i est la variable x, j est la variable y
  int tmp = i;
  i = j;
  j = tmp;
}

int main()
{
  int x = 1;
  int y = 2;
  swap( x, y ); // après, x = 2, y = 1
```

```
}
```

On peut aussi déclarer des *références non-modifiables* via le mot-clé `const`, par exemple pour passer en **entrée** un paramètre de taille importante. Il n'est ainsi pas recopié, mais vous pouvez y accéder comme si c'était une variable locale.

```
struct Personne {
  char nom[ 100 ];
  char prenom[ 100 ];
};
void affiche( const Personne & p ) // rapide: pas de copie de Personne
{
  std::cout << p.prenom << " " << p.nom << std::endl;
}
int main()
{
  Personne P1 = { "Jean", "Bon" }; // Miam...
  affiche( P1 );
}
```

On ne peut déclarer de références sans les initialiser immédiatement. Ainsi :

```
int a = 7;
int& b = a; // valide, b est un alias de a
int& c; // invalide, il faut désigner une variable du bon type.
b = 3; // à la fois a et b valent 3 (b est la variable a)
```

Puisqu'une référence désigne en fait une autre variable dès sa création, on ne peut donc changer la variable désignée par cette référence par la suite.

Les références peuvent apparaître :

1. dans un bloc de programme, pour se donner un autre nom pour une variable :

   ```
   Personne P[ 100 ]; ...
   Personne& p10 = P[ 10 ];  // p10 est la 11e personne de P
   ```

2. dans les paramètres de fonction, pour simuler des passages en entrées et/ou sorties :

   ```
   void metAuCarre( double& x ) { x *= x; }
   ```

3. comme donnée membre d'une classe ou structure ; attention à l'initialiser dans le constructeur

   ```
   struct Donnees { ... };
   struct VueDonnees {
     Donnees& myData;
     int myStart, myEnd;
     // Constructeur, requiert une Donnees
     VueDonnees( Donnees& data, int s, int e )
     : myData( data ) // nécessairement initialisé ici
     { myStart = s; myEnd = e; } // peut être fait là ou sur la ligne au-
         dessus
   };
   ```

4. comme valeur de retour d'une fonction ou d'une méthode, pour référencer une donnée membre par exemple.

   ```
   struct Donnees {
     Personne myP[ 100 ];
     // surcharge de l'opérateur [ entier ] pour accéder en écriture au
         tableau
     Personne& operator[]( int i )
     { return myP[ i ]; }
   };
   ```

Une référence ne possède pas l'objet désigné : cet objet doit rester vivant pendant son utilisation. Ne retournez jamais une référence vers une variable locale ordinaire de la fonction.

1.4.1 Références de rvalue : `T&&`.

Les références précédentes, `T&`, sont dites références de lvalue. Une expression nommant une variable est généralement une *lvalue* ; un résultat temporaire comme `std::string("bonjour")` est une *rvalue*. Une référence `T&&` peut se lier à une rvalue et permet notamment de mettre en place le *déplacement* des ressources.

```cpp
// programme complet
#include <iostream>
#include <string>
#include <utility> // std::move

void examine(const std::string&) { std::cout << "lecture\n"; }
void examine(std::string&&)      { std::cout << "rvalue\n"; }

int main() {
  std::string s = "bonjour";
  examine(s);                          // lecture
  examine(std::string("salut"));       // rvalue
  examine(std::move(s));               // rvalue ; aucun transfert ici
  std::string t = std::move(s);        // construction par deplacement
  std::cout << t << '\n';              // bonjour
  s = "reutilisable";                  // affectation valide
}
```

`std::move` ne déplace rien à lui seul : il produit une expression permettant de sélectionner une opération de déplacement, lorsqu'elle existe. Après le déplacement ci-dessus, `s` est une chaîne valide dont le contenu n'est pas spécifié ; il ne faut pas supposer qu'elle est vide. Un déplacement peut éviter une copie coûteuse, sans garantir systématiquement un gain. Depuis un objet `const`, `std::move` conduit souvent à une copie, car les opérations de déplacement attendent un objet modifiable.

Une référence de rvalue *nommée* est elle-même une expression lvalue :

```cpp
// fragment utilisant examine ci-dessus
std::string&& r = std::string("texte");
examine(r);              // lecture : l'expression r est une lvalue
examine(std::move(r));   // rvalue
```

Ici, la liaison directe à `r` prolonge la vie du temporaire jusqu'à la fin de la portée de `r`. Ce prolongement ne s'applique pas arbitrairement à tous les retours de fonction ou passages de paramètres. Les références de transmission (*forwarding references*) et `std::forward` seront étudiées avec les templates.

1.5 Fonctions

La déclaration et la définition des fonctions en C++ suivent un mécanisme très similaire à celui du C. Les fonctions sont *déclarées* sous la forme

TypeValeurDeRetour NomFonction(*TypeParam1 NomParam1*, ..., *TypeParamN NomParamN*);

Les fonctions sont *définies* sous la même forme, en rajoutant le code associé :

TypeValeurDeRetour NomFonction(*TypeParam1 NomParam1*, ..., *TypeParamN NomParamN*)
{
  .../* code */
}

On déclare en général les fonctions dans les fichiers en-tête (pour que d'autres modules puissent s'en servir), on définit en général les fonctions dans les fichiers sources (les autres modules n'ont pas besoin de savoir comment la fonction est écrite). On peut bien sûr aussi écrire des fonctions récursives, comme celle-ci :

```cpp
// mymath.hpp: Ma bibliothèque de math...
// Calcule x^n en temps log(n).
double power( double x, int n );
```

```cpp
// mymath.cpp: Voilà son secret...
// Calcule x^n en O(log(|n|+1)) ; voir les preconditions ci-dessous.
double power( double x, int n )
{
  if ( n < 0 ) return 1.0 / power( x, -n );          // x^(-n) = 1/x^n
  else if ( n == 0 ) return 1.0;                     // x^0 = 1
  else if ( n == 1 ) return x;                       // x^1 = x
  else if ( n % 2 == 0 ) return power( x * x, n / 2 ); // x^(2n) = (x^2)^n
  else return power( x * x, n / 2 ) * x;             // x^(2n+1) = ((x^2)^n)*x
}
```

Cet exemple suppose que `n` n'est pas le plus petit entier de type `int`, dont l'opposé ne serait pas représentable ; pour un exposant négatif, on suppose aussi `x` non nul.

Il est légitime en C++ de *surcharger* des fonctions, c'est-à-dire de définir des fonctions de même nom mais de signature différente (les paramètres sont différents). C'est le compilateur qui choisit la fonction la plus appropriée par rapport aux types que vous donnez en paramètres. On note aussi que le C++ fait des conversions implicites de type dans certains cas pour essayer de trouver la fonction adéquate.

```
double power( double x, int n ); // (1)
float power( float x, int n );   // (2) légitime, on peut faire une version pour
    les float
...
double x = power( 1.4, 3 ); // appelle (1)
float y  = power( 1.4f, 3 ); // appelle (2)
float z  = power( 1, 3 ); // ERREUR: le compilateur ne peut choisir entre (1) et
    (2)
```

On note que le compilateur n'utilise pas le type de la valeur de retour pour choisir quelle fonction il appelle. Le type de la valeur de retour peut varier entre fonctions surchargées.

Comme autre exemple, on peut envisager d'additionner des éléments variés. On peut donc surcharger une fonction `add` pour divers paramètres, ou même surcharger l'opérateur binaire `+`.

```
struct Point { double abs; double ord; };
double add( double a, double b )
{
  return a+b;
};
Point add( Point p1, Point p2 )
{
  Point q = { p1.abs + p2.abs, p1.ord + p2.ord };
  return q;
}
Point operator+( Point p1, Point p2 )
{
  return add( p1, p2 );
}
```

Les définitions des templates sont généralement placées dans les fichiers en-tête afin d'être accessibles au point d'instanciation. Les exceptions, notamment l'instanciation explicite, seront présentées plus loin.

1.6 Structures et classes

Comme beaucoup de langages, le C++ offre un moyen de définir de nouveaux types complexes en agrégeant les types déjà créés via des structures (enregistrements, *record*, classes). Il offre deux mot-clés très similaires pour le faire : `struct` et `class`.

Les accès aux membres et aux classes de base sont publics par défaut pour `struct`, privés pour `class`. Ainsi, par défaut, une structure est *publique*, et que donc des fonctions extérieures à la structure peuvent accéder aux champs et aux méthodes de la structure, tandis qu'une classe est par défaut *privée* et interdit à toute fonction extérieure d'accéder à ses données : seules ses méthodes ont ce droit-là. On voit donc que les deux mot-clés ont été conservés uniquement pour que les structures aient par défaut les mêmes propriétés qu'en langage C, tandis que les classes poussent le programmeur à préserver l'*encapsulation* des données.

1.6.1 Déclaration des structures et classes

On prend l'exemple standard de la définition d'un point dans le plan.

```
struct Point {
  double x;
  double y;
};
...
Point p;
p.x = 17; // valide
```

```
class Point {
  double x; // en fait privé
  double y; // en fait privé
};
...
Point p;
p.x = 17; // INVALIDE
```

Ici, nous avons simplement déclaré qu'un `Point` est l'agrégat de deux `double`. Le C++ contient donc tout le mécanisme du C de construction de types.

```cpp
// Point.hpp
#ifndef _POINT_HPP_  // évite d'inclure deux fois un fichier en-tête
#define _POINT_HPP_

class Vecteur; // déclare l'existence d'une classe Vecteur

class Point {
  double coord[ 2 ]; // privé, accessible à partir des méthodes.
public:
  Point(); // déclare un point à l'origine du repère (0,0)
  Point( double aX, double aY ); // déclare un point (aX,aY)
  double x() const; // const = indique que la méthode ne
  double y() const; // change pas les données membres.
  // crée un nouveau point translaté (donc méthode const)
  Point operator+( const Vecteur & v ) const;
  // Translate ce point du vecteur spécifié (donc méthode non-const)
  Point& operator+=( const Vecteur & v );
};
#endif
```

Listing 1 – Fichier Point.hpp

1.6.2 Méthodes dans les classes ; surcharge d'opérateurs

Le C++ rajoute la possibilité de définir des fonctions attachées à une classe ou à une instance de cette classe : on parle de *méthode*. On retrouve le principe de la programmation objet en C++. On note aussi l'existence de méthodes particulières :

- les *constructeurs*, qui portent le nom de la classe (mettons `T`). Le constructeur adéquat est appelé lors de l'instanciation d'une variable de ce type, autrement dit lors de la création de l'*objet*.
- le *destructeur*, qui porte le nom `~T`. Le destructeur est appelé juste avant la disparition de la variable de ce type.

On peut aussi redéfinir l'opérateur d'affectation (`operator=`), mais aussi quasiment tous les opérateurs usuels pour les adapter à la classe.

On donne sur les Listings 1, 2, 3, 4 et 5, un exemple assez complet de création de classes `Point` et `Vecteur` du plan. Il y a beaucoup de choses à comprendre dans ces exemples :

- différence entre déclaration et définition d'une classe
- une classe seulement déclarée (ici `Vecteur`) n'est accessible que par référence ou pointeur (sinon le compilateur doit connaître sa taille).
- les méthodes peuvent être `const` ou non ; les premières interdisent la modification directe des membres non `mutable` via `this` ; il ne s'agit pas d'une garantie d'absence de tout effet de bord.
- lorsqu'une méthode est définie dans le fichier source, il faut rappeler qu'elle appartient à la classe `T` en écrivant `T::` devant le nom de la méthode.
- le pointeur `this`, dans une méthode, est un pointeur vers l'instance actuelle, i.e. c'est l'adresse de l'objet en mémoire.
- on peut redéfinir les opérateurs arithmétiques et logiques usuels pour les adapter à chaque classe. C'est très pratique quand on modélise des objets mathématiques.

Définir une méthode dans la classe la rend implicitement `inline` dans les en-têtes ordinaires. Ce mot-clé autorise notamment certaines définitions identiques dans plusieurs unités de traduction ; il ne garantit pas que le compilateur remplacera chaque appel par le corps de la fonction.

1.6.3 Initialisation des données membres.

Une *liste d'initialisation* construit les membres avant l'exécution du corps du constructeur. Elle existe depuis C++98. Affecter une valeur dans le corps intervient après leur initialisation, ce qui n'est pas équivalent. Depuis C++11, une valeur par défaut peut aussi être indiquée dans la classe.

```cpp
// classe complete
#include <string>
#include <utility>
```

```cpp
// Point.cpp
#include "Point.hpp"    // on a besoin des déclarations de
#include "Vecteur.hpp" // ces deux classes

Point::Point()
{
  coord[ 0 ] = 0.0; // initialisation forcée à zéro des membres.
  coord[ 1 ] = 0.0;
}
Point::Point( double aX, double aY )
{
  coord[ 0 ] = aX;
  coord[ 1 ] = aY;
}
double Point::x() const
{
  return coord[ 0 ];
}
double Point::y() const
{
  return coord[ 1 ];
}
Point Point::operator+( const Vecteur & v ) const
{ // Crée directement le point avec les bonnes coordonnées
  return Point( x() + v.x(), y() + v.y() );
}
Point& Point::operator+=( const Vecteur & v )
{ // Crée directement le point avec les bonnes coordonnées
  coord[ 0 ] += v.x();
  coord[ 1 ] += v.y();
  return *this; // retourne une référence à soi-même
}
```

Listing 2 – Fichier Point.cpp

```cpp
// Vecteur.hpp
#ifndef _VECTEUR_HPP_
#define _VECTEUR_HPP_

class Vecteur {
  double comp[ 2 ];
public:
  Vecteur(); // déclare le vecteur nul (0,0)
  Vecteur( double u, double v ); // déclare un vecteur (u,v)
  double x() const; // const = indique que la méthode ne
  double y() const; // change pas les données membres.
  // les vecteurs s'additionnent
  Vecteur operator+( const Vecteur & v ) const; // v3 = v1 + v2
  Vecteur& operator+=( const Vecteur & v );     // v2 += v1
  // les vecteurs se multiplie par un scalaire
  Vecteur operator*( double k ) const; // permet v2 = v1 * k
  Vecteur& operator*=( double k );     // permet v1 *= k
};
Vecteur operator*( double k, const Vecteur & v ); // permet v2 = k * v1

#endif
```

Listing 3 – Fichier Vecteur.hpp

```cpp
// Vecteur.cpp
#include "Vecteur.hpp"

// déclare le vecteur nul (0,0)
Vecteur::Vecteur()
{
  comp[ 0 ] = comp[ 1 ] = 0.0;
}
// déclare un vecteur (u,v)
Vecteur::Vecteur( double u, double v )
{
  comp[ 0 ] = u;
  comp[ 1 ] = v;
}
// accesseur à la composante x
double Vecteur::x() const
{
  return comp[ 0 ];
}
// accesseur à la composante y
double Vecteur::y() const
{
  return comp[ 1 ];
}
// les vecteurs s'additionnent (v3 = v1 + v2)
Vecteur Vecteur::operator+( const Vecteur & v ) const
{
  return Vecteur( x() + v.x(), y() + v.y() );
}
// Permet v2 += v1
Vecteur& Vecteur::operator+=( const Vecteur & v )
{
  comp[ 0 ] += v.x();
  comp[ 1 ] += v.y();
  return *this;
}
// les vecteurs se multiplie par un scalaire (v2 = v1 * k)
Vecteur Vecteur::operator*( double k ) const
{
  return Vecteur( k * x(), k * y() );
}
// permet v1 *= k
Vecteur& Vecteur::operator*=( double k )
{
  comp[ 0 ] *= k;
  comp[ 1 ] *= k;
  return *this;
}
// On note qu'il s'agit d'une faction, pas d'une méthode de la classe
// Vecteur. Permet v2 = k * v1
Vecteur operator*( double k, const Vecteur & v )
{
  return Vecteur( k * v.x(), k * v.y() );
}
```

Listing 4 – Fichier Vecteur.cpp

```cpp
#include <iostream>
#include "Point.hpp"
#include "Vecteur.hpp"

int main()
{
  Point p( 2, 1 );
  Vecteur u( 3, 1 );
  p += u;
  std::cout << "p=(" << p.x() << "," << p.y() << ")"
            << std::endl; // affiche p=(5,2)
  u *= 3; // u=(9,3)
  Point q = p + u;
  std::cout << "q=(" << q.x() << "," << q.y() << ")"
            << std::endl; // affiche q=(14,5)
  Vecteur v = -1 * u; // v=(-9,-3)
  q += 2 * v; // appelle operator*(double,Vecteur) puis Point::operator+=
  std::cout << "q=(" << q.x() << "," << q.y() << ")"
            << std::endl; // affiche q=(-4,-1)
  return 0;
  // les destructeurs de p, u, q, et v sont automatiquement appelés.
}
```

Listing 5 – Fichier point-vecteur.cpp

```cpp
class Mesure {
  std::string nom{"sans nom"}; // initialiseur de membre par defaut
  double valeur{0.0};
public:
  Mesure() = default;
  Mesure(std::string n, double v)
    : nom(std::move(n)), valeur(v) // liste d'initialisation
  {}
  double lire() const { return valeur; }
};
```

Un initialiseur présent dans la liste du constructeur remplace l'initialiseur par défaut du membre concerné. Les membres sont initialisés dans leur *ordre de déclaration*, quel que soit l'ordre écrit dans la liste. Les références et les membres `const` doivent être initialisés, non affectés dans le corps.

C++20 ajoute les *initialiseurs désignés* pour les agrégats, par exemple une structure sans constructeur déclaré par l'utilisateur et sans membres privés :

```cpp
// C++20 ; fragment
struct Options {
  int largeur{640};
  int hauteur{480};
  bool plein_ecran{false};
};
Options a{.largeur = 800, .hauteur = 600};
Options b{.hauteur = 720}; // largeur et plein_ecran gardent leurs defauts
```

Les membres désignés doivent suivre l'ordre de déclaration. Cette syntaxe initialise un agrégat ; ce n'est ni un appel de constructeur à paramètres nommés, ni une affectation à un objet existant.

1.6.4 Surcharge des opérateurs flux

On peut aussi surcharger les opérateurs de flux de sortie (`operator«`) et d'entrée (`operator»`), afin de faciliter la sérialisation ou l'affichage d'un objet. On rajouterait par exemple dans le fichier `Point.hpp` les lignes suivantes :

```cpp
  inline std::ostream& operator<<( std::ostream & out, const Point & p )
  {
    out << "(" << p.x() << "," << p.y() << ")";
    return out;
  }
```

On pourrait alors réécrire le Listing 5 sous la forme plus compacte :

```
...
p += u;
std::cout << "p=" << p << std::endl; // affiche p=(5,2)
u *= 3; // u=(9,3)
Point q = p + u;
std::cout << "q=" << q << std::endl; // affiche q=(14,5)
Vecteur v = -1 * u; // v=(-9,-3)
q += 2 * v; // appelle operator*(double,Vecteur) puis Point::operator+=
std::cout << "q=" << q << std::endl; // affiche q=(-4,-1)
...
```

On note surtout que l'on utilisera de la même façon les flux pour afficher, pour sauvegarder dans un fichier, ou pour créer une chaîne de caractères.

1.7 Espaces de noms : namespace

Le C++ offre un autre moyen logique pour classer les différentes définitions de variables, fonctions, classes. Ce mécanisme s'appelle les *espaces de noms*, via le mot-clé `namespace`. Cela permet notamment de ne pas utiliser par inadvertance une fonction/classe définie quelque part, tout en pensant en fait utiliser une autre.

Ainsi, par défaut, toutes vos définitions sont dans l'espace de nommage global. En revanche, toutes les variables, fonctions et classes de la bibliothèque standard C++ sont rangées dans l'espace de nom `std`. Pour accéder à celles-ci, il faut donc utiliser l'opérateur de résolution de portée `::`, qui permet de spécifier où le compilateur doit chercher l'identifiant.

Cela explique pourquoi on écrit `std::cout` pour accéder à la variable globale `cout` placée dans l'espace de noms `std`.

Vous pouvez vous-même définir vos fonctions/classes dans votre propre espace de noms, par exemple :

```
namespace heroes {
  class Superman { ... };
  class Batman { ... };
  class Bicyclerepairman { ... };
}
...
heroes::Superman clark_kent;
heroes::Batman bruce_wayne;
heroes::Bicyclerepairman michael_palin;
```

Dans un fichier source (mais ce **n'est pas** recommandé dans un fichier en-tête), on peut utiliser la directive `using namespace` pour que le compilateur recherche aussi dans cet espace de noms les identifiants spécifiés. Sur l'exemple précédent, on aurait pu écrire :

```
using namespace heroes;
...
Superman clark_kent;
Batman bruce_wayne;
Bicyclerepairman michael_palin;
```

1.8 Chaînes de caractères

1.8.1 Manipuler une `std::string`.

Les chaînes du C sont des tableaux de caractères terminés par un caractère nul (`'\0'`). La classe `std::string`, déclarée dans `<string>`, gère sa mémoire et sa taille automatiquement. Elle peut contenir des caractères nuls internes. Elle ne protège toutefois pas contre un accès hors limites effectué avec `operator[]`.

```
// programme complet
#include <iostream>
#include <string>
int main() {
  std::string nom = "Ada";
  std::string message = "Bonjour " + nom;
  message += " !";
  std::cout << message << '\n';
```

```
  std::cout << message.size() << '\n';
  std::cout << message.substr(0, 7) << '\n'; // Bonjour
  std::string::size_type pos = message.find(nom);
  if (pos != std::string::npos)
    std::cout << "Position : " << pos << '\n';
  message.at(0) = 'b'; // acces verifie ; peut lever std::out_of_range
}
```

`find` renvoie `std::string::npos` en cas d'échec. La méthode `c_str()` fournit un pointeur vers une représentation terminée par zéro pour une API C ; ne conservez pas ce pointeur après destruction ou après une opération susceptible de réallouer la chaîne.

```
// C++20 ; fragment ; #include <string>
std::string fichier = "rapport.txt";
bool est_rapport = fichier.starts_with("rapport");
bool est_texte = fichier.ends_with(".txt");
```

`contains` pour rechercher une sous-chaîne n'arrive qu'en C++23 ; en C++20, utilisez `find`.

1.8.2 Observer sans copier : `std::string_view` (C++17).

Une vue représente une séquence de caractères sans posséder sa mémoire. Elle évite une copie, mais les caractères observés doivent rester vivants. La séquence n'est pas nécessairement terminée par zéro.

```
// C++17 ; programme complet
#include <iostream>
#include <string>
#include <string_view>
void affiche(std::string_view texte) {
  std::cout << texte << '\n';
}
int main() {
  std::string nom = "Ada Lovelace";
  affiche(nom);
  affiche(std::string_view(nom).substr(0, 3)); // Ada
  // std::string_view mauvaise = std::string("temporaire");
  // Invalide a l'utilisation : l'objet temporaire serait deja detruit.
}
```

1.8.3 Unicode, UTF-8 et internationalisation.

Un caractère perçu par un lecteur, un point de code Unicode et une unité de codage ne sont pas la même chose. UTF-8 représente un point de code par une à quatre unités de huit bits. Un graphème peut réunir plusieurs points de code, par exemple une lettre suivie d'un accent combinant.

`std::string` stocke des `char` sans imposer de codage. Lorsqu'on y stocke de l'UTF-8 sur une machine à octets de huit bits, `size()` compte les octets, et `operator[]` accède à un octet, non à un caractère Unicode. Découper à une position arbitraire peut donc produire une séquence UTF-8 invalide.

```
// Programme complet ; compatible jusqu'a C++17 inclus
#include <iostream>
#include <string>
int main() {
  std::string texte = u8"caf\u00e9"; // cafe avec e accent aigu
  std::cout << texte.size() << '\n'; // 5 unites UTF-8, 4 points de code
}
```

Cet exemple emploie volontairement un échappement Unicode, de sorte que le code source reste ASCII. Un littéral ordinaire comme `"café"` dépend du codage des littéraux choisi par l'implémentation et ses options.

Depuis C++20, les littéraux `u8` utilisent le type distinct `char8_t` ; on les stocke naturellement dans une `std::u8string` :

```
// C++20 ; programme complet
#include <iostream>
#include <string>
int main() {
  std::u8string texte = u8"caf\u00e9";
```

```
  std::cout << texte.size() << '\n'; // 5
  // std::string s = u8"caf\u00e9"; // invalide en C++20
  // std::cout << texte;            // pas de surcharge standard adaptee
}
```

`std::u8string` rend l'intention de codage explicite, mais ne valide pas son contenu et ne fournit pas à elle seule les opérations Unicode. `std::u16string` et `std::u32string` existent depuis C++11 ; `std::wstring` ne garantit pas un codage Unicode portable.

L'internationalisation comprend aussi la traduction des messages, le tri, les conversions de casse, la normalisation et les formats locaux de nombres et de dates. `std::locale` permet certaines adaptations, mais ne constitue pas un moteur Unicode complet ; appliquer `std::toupper` aux octets d'une chaîne UTF-8 n'effectue pas une conversion Unicode. Pour ces traitements, on peut utiliser une bibliothèque spécialisée telle qu'ICU. En C++20, il n'existe pas de mécanisme standard complet de segmentation en graphèmes et de normalisation Unicode. La console, les fichiers et les bibliothèques doivent aussi s'accorder sur le codage des échanges.

1.9 Mettre en forme du texte avec `std::format` (C++20)

L'en-tête `<format>` fournit `std::format`, qui construit une chaîne sans l'afficher. Les champs entre accolades reçoivent les arguments ; un deux-points introduit les options de présentation.

```
// C++20 ; programme complet
#include <format>
#include <iostream>
#include <string>
int main() {
  std::string nom = "Ada";
  double x = 3.141592653589793;
  std::string message = std::format("Bonjour {}, x = {:.3f}", nom, x);
  std::cout << message << '\n'; // Bonjour Ada, x = 3.142
  std::cout << std::format("{:04d} | {:>8} | {:#x}", 7, nom, 255)
            << '\n'; // 0007 |      Ada | 0xff
  std::cout << std::format("{{valeur}} = {}", 42) << '\n';
}
```

`.3f` impose trois décimales, `04d` une largeur minimale de quatre caractères complétée par des zéros, `>8` un alignement à droite et `#x` une écriture hexadécimale préfixée. Les doubles accolades produisent des accolades littérales. Les types des arguments sont pris en compte ; les appels usuels avec une chaîne de format littérale sont vérifiés à la compilation par les bibliothèques conformes aux corrections du standard.

Pour un format construit à l'exécution, utilisez `std::vformat` ; un format invalide peut lever `std::format_error` :

```
// C++20 ; fragment ; #include <format> et <string>
std::string motif = "valeur = {:.2f}";
double x = 1.25;
std::string resultat = std::vformat(motif, std::make_format_args(x));
```

Par défaut, le formatage des nombres est indépendant de la locale. Une surcharge recevant une `std::locale` et l'option `L` permet une présentation localisée :

```
// C++20 ; fragment ; #include <format>, <locale> et <string>
std::string resultat = std::format(std::locale(""), "{:L}", 1234567);
```

La locale vide demande celle de l'environnement ; sa disponibilité dépend du système et sa construction peut échouer. `std::format` ne traduit pas les messages. Depuis C++23, `std::print` et `std::println` (en-tête `<print>`) permettent un affichage direct avec une syntaxe similaire. Le support dépend aussi de la version de la bibliothèque standard.

1.10 Mesurer le temps avec `std::chrono`

La bibliothèque `<chrono>` distingue les horloges, les instants (`time_point`) et les durées (`duration`). Soustraire deux instants d'une même horloge fournit une durée. Pour mesurer un intervalle, préférez `steady_clock`, qui est monotone. `system_clock` sert au temps civil et peut être ajustée. L'horloge `high_resolution_clock` n'est pas nécessairement monotone.

```cpp
// programme complet
#include <chrono>
#include <iostream>
#include <thread>
int main() {
  const auto debut = std::chrono::steady_clock::now();
  std::this_thread::sleep_for(std::chrono::milliseconds(20));
  const auto fin = std::chrono::steady_clock::now();
  const std::chrono::duration<double, std::milli> ms = fin - debut;
  const auto ms_entieres =
    std::chrono::duration_cast<std::chrono::milliseconds>(fin - debut);
  std::cout << ms.count() << " ms\n";
  std::cout << ms_entieres.count() << " ms entieres\n";
}
```

Le résultat flottant conserve une partie fractionnaire ; la conversion vers une durée entière tronque vers zéro. La période choisie n'augmente pas la précision de l'horloge. `sleep_for` peut attendre plus que demandé. Remplacez cet appel par le traitement à mesurer ; pour un calcul très court, répétez les mesures et utilisez effectivement le résultat du calcul afin de limiter les effets de l'optimisation. Il s'agit ici de temps écoulé, pas de temps CPU consommé par le processus.

```cpp
// C++14 ; fragment ; #include <chrono>
using namespace std::chrono_literals;
auto attente = 250ms;
auto total = 1s + attente; // duree de 1250 millisecondes
```

C++20 complète `chrono` avec des dates calendaires et des fuseaux horaires, à distinguer de la mesure d'un intervalle.

1.11 Manipuler les chemins avec `std::filesystem` (C++17)

Un chemin n'est pas simplement un nom de fichier : il peut comporter des répertoires, une racine et une extension. `std::filesystem::path` (en-tête `<filesystem>`) représente ces éléments et adapte les séparateurs à la plateforme. L'opérateur `/` assemble les composantes.

```cpp
// C++17 ; programme complet
#include <filesystem>
#include <iostream>
int main() {
  namespace fs = std::filesystem;
  fs::path p = fs::path("donnees") / "mesure.csv";
  std::cout << p.filename().string() << '\n';     // mesure.csv
  std::cout << p.stem().string() << '\n';         // mesure
  std::cout << p.extension().string() << '\n';    // .csv
  std::cout << p.parent_path().string() << '\n'; // donnees
  fs::path sortie = p;
  sortie.replace_extension(".txt"); // modifie sortie, pas p
  std::cout << sortie.string() << '\n';
  fs::path q = fs::path("donnees") / ".." / "images" / "photo.png";
  std::cout << q.lexically_normal().string() << '\n';
}
```

Ces opérations sont lexicales et ne nécessitent pas que les fichiers existent. `extension()` ne renvoie que la dernière extension : pour `archive.tar.gz`, `stem()` vaut `archive.tar` et `extension()` vaut `.gz`. Pour `.bashrc`, l'extension est vide. Assembler un chemin absolu à droite de `/` peut remplacer celui de gauche ; l'opérateur ne se limite donc pas à concaténer des chaînes.

Les opérations qui interrogent le système peuvent échouer. Les surcharges ordinaires signalent les erreurs du système par `filesystem_error` ; d'autres reçoivent un `std::error_code` :

```cpp
// C++17 ; programme complet
#include <filesystem>
#include <iostream>
#include <system_error>
int main() {
  namespace fs = std::filesystem;
  std::error_code erreur;
  const bool existe = fs::exists("donnees/mesure.csv", erreur);
  if (erreur)
    std::cerr << erreur.message() << '\n';
```

```
  else
    std::cout << (existe ? "Existe\n" : "Absent\n");
}
```

La consultation ne garantit pas que le fichier sera toujours présent à l'ouverture. `current_path`, `is_regular_file` et `directory_iterator` permettent respectivement d'obtenir le répertoire courant, de tester un fichier ordinaire et de parcourir un dossier. `canonical` consulte le système et résout les liens symboliques ; `lexically_normal` ne le fait pas et peut donc donner un résultat différent en présence de liens symboliques.

Le codage natif des chemins dépend du système : `p.string()` n'est pas garanti UTF-8. Depuis C++20, on peut construire un chemin à partir d'un littéral `u8` et en obtenir une représentation UTF-8 typée :

```
// C++20 ; fragment ; #include <filesystem> et <string>
std::filesystem::path p = u8"donn\u00e9es/r\u00e9sultat.txt";
std::u8string texte = p.u8string();
```

En C++17, `std::filesystem::u8path` permet de construire un chemin à partir d'une chaîne UTF-8 de `char` ; cette fonction est dépréciée depuis C++20 au profit des entrées de type `char8_t`.

1.12 Complément : catégories de valeurs

Les catégories décrivent les *expressions*, et non le type des objets. Depuis C++11, trois catégories fondamentales sont distinguées :

- une *lvalue* désigne une identité, par exemple une variable nommée, `*p`, `t[i]` ou `++i` avec les opérateurs usuels ;
- une *prvalue* calcule une valeur ou initialise un objet, par exemple `42`, `i + 1` ou `std::string("texte")` ;
- une *xvalue* désigne un objet dont les ressources peuvent être réutilisées, comme avec `std::move(s)`.

Les *rvalues* réunissent les prvalues et les xvalues ; les *glvalues* réunissent les lvalues et les xvalues. Une lvalue n'est pas nécessairement modifiable : une variable `const` est aussi une lvalue. Le sens historique « à gauche ou à droite d'une affectation » ne suffit donc pas à définir ces catégories.

```
// fragment ; #include <utility>
int n = 7;
int& a = n;                 // reference de lvalue
const int& b = 12;          // liaison a un temporaire, vie prolongee
int&& c = 12;               // reference de rvalue, vie prolongee
// int& erreur = 12;        // invalide
int&& d = std::move(n);     // d designe n ; aucun nouvel entier cree
a = 3;                      // modifie n
c = 5;                      // c est une expression lvalue
```

Avec l'opérateur d'adresse intégré, une lvalue d'objet adressable permet habituellement de prendre son adresse. Ce n'est pas un critère universel : un champ de bits, par exemple, peut être une lvalue sans que son adresse puisse être prise. Ces distinctions seront utiles pour les constructeurs de déplacement et la transmission parfaite des arguments.

Références techniques. Les descriptions précises figurent dans le projet public du standard C++, notamment les sections `[basic.lval]`, `[lex.string]`, `[format]`, `[time.clock.steady]` et `[filesystems]` : `https://eel.is/c++draft/`. Ce projet évolue ; les indications de version ci-dessus distinguent les fonctionnalités présentées dans ce chapitre.

2 Programmation orientée objet, redéfinition et surcharge

Le C++ offre les mécanismes classiques de la programmation orientée objet : encapsulation, membres d'instance ou de classe, héritage simple et multiple, classes abstraites et polymorphisme dynamique. Les types peuvent aussi être interrogés à l'exécution grâce au RTTI (*run-time type information*). Contrairement à Python, les définitions des méthodes ne sont pas remplaçables à l'exécution par les mécanismes ordinaires du langage.

Dans ce chapitre, les exemples sans indication de version fonctionnent en C++11. Une version minimale n'est précisée que si elle est postérieure. Les fragments illustratifs peuvent contenir des omissions ; les programmes complets sont signalés comme tels.

2.1 Encapsulation

Les membres d'une classe ou d'une structure ont trois niveaux d'accès :

public : accessibles au code extérieur comme au code de la classe ; c'est l'accès par défaut d'une `struct`.

protected : accessibles à la classe et, sous les règles d'accès de l'héritage, aux classes dérivées ; ils ne constituent pas une interface publique.

private : accessibles à la classe elle-même, mais pas directement aux classes dérivées ; c'est l'accès par défaut d'une `class`.

Les fonctions et classes amies (`friend`) disposent aussi des accès qui leur sont accordés. Un niveau d'accès s'introduit avec un deux-points.

```
// Fragment illustratif
class A {
public:
  A() = default;
  int accessible{0};
protected:
  int protege{0};
private:
  int prive{0};
};
class B : public A {
public:
  B() {
    accessible = 4;
    protege = 2;
    // prive = 0; // invalide
  }
};
// Dans une fonction :
// A a;
// a.accessible = 5; // valide
// a.protege = 3;    // invalide
```

L'encapsulation sert notamment à préserver les invariants : rendre une donnée privée permet de contrôler ses modifications par une interface. Ajouter systématiquement un accesseur et un modificateur à chaque donnée ne suffit pas à garantir cette protection.

2.2 Membres d'instance et membres de classe

Un membre non statique appartient à chaque objet. Un membre `static` est associé à la classe : une fonction statique n'a pas de pointeur `this`, et une donnée statique est partagée entre les instances. Voici un compteur d'objets vivants, y compris ceux construits par copie ou par déplacement :

```
// Fichier Compteur.hpp
struct Compteur {
  Compteur() { ++nb; }
  Compteur(const Compteur&) { ++nb; }
  Compteur(Compteur&&) noexcept { ++nb; }
  Compteur& operator=(const Compteur&) = default;
  Compteur& operator=(Compteur&&) noexcept = default;
  ~Compteur() { --nb; }
  static int nbInstances() { return nb; }
private:
  static int nb;
  // inline static int nb{ 0 }; // depuis C++17
};
```

```
// Avant C++17: Fichier Compteur.cpp
#include "Compteur.hpp"
int Compteur::nb = 0;
```

```cpp
// Fichier main.cpp, a lier avec Compteur.cpp
#include <iostream>
#include <utility> // std::move
#include "Compteur.hpp"
int main() {
  Compteur a;
  Compteur b = a; // copie : deux objets vivants
  {
    Compteur c(std::move(b)); // deplacement : b reste vivant
    std::cout << Compteur::nbInstances() << '\n'; // 3
    c = std::move(a); // affectation par deplacement : toujours 3
    std::cout << Compteur::nbInstances() << '\n'; // 3
  } // destruction de c
  b = a; // affectation par copie : aucun nouvel objet
  std::cout << Compteur::nbInstances() << '\n';   // 2
}
```

Le constructeur par déplacement prend une référence de rvalue (`Compteur&&`) et crée un nouvel objet : il incrémente donc le compteur, comme le constructeur par copie. L'objet source reste vivant et sera détruit normalement. Les affectations, par copie ou par déplacement, modifient un objet existant : elles ne doivent ni copier le compteur ni l'incrémenter. Ici, il n'y a aucune ressource à transférer, seulement des instances à compter. Cet exemple n'est pas prévu pour des constructions simultanées depuis plusieurs threads. Depuis C++17, `inline static int nb{0};` permet de définir le membre directement dans la classe, sans définition séparée dans `Compteur.cpp`.

2.3 Sous-typage et héritage simple

L'héritage public modélise une relation « est un » : un objet dérivé peut être utilisé à travers l'interface de sa base. Cette relation doit rester cohérente avec les comportements attendus de la base. Les héritages privé et protégé servent plutôt à partager une implémentation.

L'exemple suivant est un modèle pédagogique simplifié, non une taxonomie biologique exhaustive : les nombres de membres décrivent ici les espèces représentées. `Animal` définit un comportement commun, `Mammifere` ajoute le cri et `Insecte` fixe à six le nombre de pattes. Le nom seul ne justifierait pas cette hiérarchie.

```cpp
// Definitions communes aux exemples d'animaux ci-dessous
#include <string>

class Animal {
  std::string mNom;
public:
  explicit Animal(const std::string& nom) : mNom(nom) {}
  virtual ~Animal() = default;
  const std::string& nom() const { return mNom; }
  virtual int nombreMembres() const = 0;
};
class Mammifere : public Animal {
public:
  explicit Mammifere(const std::string& nom) : Animal(nom) {}
  virtual std::string cri() const = 0;
};
class Insecte : public Animal {
public:
  explicit Insecte(const std::string& nom) : Animal(nom) {}
  int nombreMembres() const override { return 6; }
};
class Lion : public Mammifere {
public:
  explicit Lion(const std::string& nom) : Mammifere(nom) {}
  int nombreMembres() const override { return 4; }
  std::string cri() const override { return "rugissement"; }
};
class Chat : public Mammifere {
public:
  explicit Chat(const std::string& nom) : Mammifere(nom) {}
  int nombreMembres() const override { return 4; }
  std::string cri() const override { return "miaulement"; }
```

```
};
class Mouche : public Insecte {
public:
  explicit Mouche(const std::string& nom) : Insecte(nom) {}
};
```

La classe de base est aussi appelée classe mère ; la classe dérivée, classe fille. Le tableau indique l'accès aux membres de la base depuis la classe dérivée ; les membres privés restent dans l'objet, même lorsqu'ils sont inaccessibles directement.

Héritage	Membre privé	Membre protégé	Membre public
`private`	inaccessible	privé	privé
`protected`	inaccessible	protégé	protégé
`public`	inaccessible	protégé	public

La base est construite avant les membres propres de la classe dérivée. Le constructeur dérivé sélectionne son constructeur dans sa liste d'initialisation, comme Lion(nom): Mammifere(nom) ci-dessus. Les constructeurs ne sont pas virtuels ; on peut hériter certains constructeurs avec une déclaration `using`, mais on ne les redéfinit pas comme des méthodes virtuelles.

2.4 Polymorphisme dynamique et redéfinition

Le polymorphisme dynamique permet de manipuler des objets de classes différentes à travers une même interface, tout en appelant le comportement correspondant au type réel de l'objet. On distingue le *type statique*, connu du compilateur, du *type dynamique*, celui de l'objet le plus dérivé.

```
// A ajouter apres les definitions d'animaux
#include <iostream>
void affiche(const Animal& a) {
  std::cout << a.nom() << " : " << a.nombreMembres()
            << " membres\n";
}
int main() {
  Lion lion("Mufasa");
  Mouche mouche("Patouche");
  affiche(lion);    // Mufasa : 4 membres
  affiche(mouche); // Patouche : 6 membres
  const Mammifere& m = lion;
  std::cout << m.cri() << '\n'; // rugissement
}
```

Dans `affiche`, `a` a pour type statique `const Animal&`. Le choix de l'implémentation de `nombreMembres()` dépend de son type dynamique. La méthode `nom()`, non virtuelle, utilise la même implémentation pour tous les animaux.

2.4.1 Méthodes virtuelles, classes abstraites, `override`.

Le mot-clé `virtual` introduit une méthode virtuelle ; `= 0` la déclare *virtuelle pure*. Une classe qui conserve au moins une méthode virtuelle pure est abstraite : elle ne peut pas être instanciée. Une classe dérivée ne devient concrète que lorsqu'elle fournit toutes les redéfinitions nécessaires. Ici, `Animal` et `Mammifere` sont abstraites ; `Lion`, `Chat`, `Insecte` et `Mouche` sont concrètes.

Écrivez `override` sur une redéfinition : le compilateur vérifie la correspondance du nom, des types des paramètres et des qualifications (notamment `const`). Le nom des paramètres n'intervient pas. Répéter `virtual` dans la classe dérivée n'est pas nécessaire.

```
// Fragment illustratif
struct Base {
  virtual ~Base() = default;
  virtual int valeur() const { return 0; }
};
struct Derivee : Base {
  int valeur() const override { return 1; }
  // int valeur() override; // invalide : const manque
};
```

Le mot-clé `final` interdit une redéfinition supplémentaire d'une méthode virtuelle, ou tout héritage d'une classe lorsqu'il suit son nom.

Le type de retour ne peut pas varier librement : la *covariance* autorise, sous certaines conditions, un pointeur ou une référence vers un type dérivé à la place du pointeur ou de la référence vers la base.

```
// Fragment illustratif : covariance
struct Copiable {
  virtual ~Copiable() = default;
  virtual Copiable* clone() const = 0;
};
struct Valeur : Copiable {
  Valeur* clone() const override { return new Valeur(*this); }
};
```

Le résultat de ce `clone` doit être pris en charge par l'appelant. La covariance ne s'étend pas à `std::unique_ptr<Valeur>` et `std::unique_ptr<Copiable>` : avec des pointeurs intelligents, les redéfinitions doivent conserver le type de retour de l'interface.

2.4.2 Destruction et collections polymorphes.

Si un objet dérivé doit être détruit via un pointeur vers sa base, celle-ci doit posséder un destructeur virtuel public dans l'usage ordinaire présenté ici. Sans lui, une telle destruction a un comportement indéfini. `Animal` possède donc `virtual ~Animal()= default;`.

Un conteneur d'objets de base ne préserve pas les objets dérivés : une copie par valeur ne copie que la partie base, phénomène appelé *découpage* (*slicing*). Ici, `Animal` étant abstraite, sa copie comme objet autonome est directement interdite. Utilisez des pointeurs pour une collection hétérogène. Un `std::unique_ptr` exprime la propriété exclusive et libère automatiquement l'objet :

```
// C++14 ; a ajouter apres les definitions d'animaux
#include <iostream>
#include <memory>
#include <vector>
int main() {
  std::vector<std::unique_ptr<Animal>> animaux;
  animaux.push_back(std::make_unique<Lion>("Mufasa"));
  animaux.push_back(std::make_unique<Chat>("Toulouse"));
  animaux.push_back(std::make_unique<Mouche>("Patouche"));
  for (const auto& a : animaux)
    std::cout << a->nom() << " : " << a->nombreMembres() << '\n';
} // destruction automatique des trois objets derives
```

`std::unique_ptr` existe en C++11 ; `std::make_unique` apparaît en C++14. Les détails de propriété et d'allocation seront repris dans la partie consacrée à la mémoire.

L'allocation dynamique n'est pas indispensable au polymorphisme :

```
// Fragment utilisant les definitions d'animaux
Lion lion("Mufasa");
Mouche mouche("Patouche");
Animal* animaux[] = {&lion, &mouche}; // pointeurs non proprietaires
for (const Animal* a : animaux) {
  // utiliser a->nom() et a->nombreMembres()
}
```

Ces pointeurs ne doivent pas survivre aux objets locaux. Il n'existe pas de tableau de références en C++, mais un conteneur de `std::reference_wrapper` peut représenter une collection de références non propriétaires. Un objet qui doit survivre à un appel peut aussi être retourné par valeur : sa durée de vie souhaitée n'impose pas, à elle seule, un appel à `new`.

Le dispatch virtuel est souvent réalisé par des tables et des pointeurs internes, sans que le standard impose cette représentation ni une taille fixe de surcoût. Le compilateur peut parfois résoudre l'appel directement. Pendant la construction ou la destruction, les appels virtuels se limitent à la classe dont le constructeur ou le destructeur s'exécute, et non aux redéfinitions d'une classe plus dérivée.

2.4.3 Autre application : formes géométriques.

Une interface graphique peut définir une interface `Component` avec des méthodes virtuelles `draw` et `onEvent`, puis des boutons, labels et conteneurs. De même, une forme géométrique peut fournir une boîte englobante sans exposer sa représentation concrète :

```cpp
// Programme complet
#include <algorithm>
#include <iostream>
struct Point { double x, y; };
struct Boite {
  Point bg, hd;
  bool contient(Point p) const {
    return bg.x <= p.x && p.x <= hd.x
        && bg.y <= p.y && p.y <= hd.y;
  }
};
class Forme {
public:
  virtual ~Forme() = default;
  virtual Boite boiteEnglobante() const = 0;
};
class Disque : public Forme {
  Point centre;
  double rayon; // precondition du constructeur : r >= 0
public:
  Disque(Point c, double r) : centre(c), rayon(r) {}
  Boite boiteEnglobante() const override {
    return {{centre.x - rayon, centre.y - rayon},
            {centre.x + rayon, centre.y + rayon}};
  }
};
class Triangle : public Forme {
  Point p1, p2, p3;
public:
  Triangle(Point a, Point b, Point c) : p1(a), p2(b), p3(c) {}
  Boite boiteEnglobante() const override {
    return {{std::min({p1.x, p2.x, p3.x}),
             std::min({p1.y, p2.y, p3.y})},
            {std::max({p1.x, p2.x, p3.x}),
             std::max({p1.y, p2.y, p3.y})}};
  }
};
int main() {
  Disque disque({-2, 1}, 3);
  Triangle triangle({-4, -3}, {1, -7}, {2, -4});
  const Forme* formes[] = {&disque, &triangle};
  Point clic{0, 0};
  for (const Forme* f : formes)
    std::cout << f->boiteEnglobante().contient(clic) << '\n';
} // affiche 1 puis 0
```

Être dans la boîte englobante est une condition nécessaire, mais pas suffisante, pour appartenir à la forme.

2.5 Surcharge, redéfinition et masquage

La *surcharge* propose plusieurs fonctions de même nom avec des types ou des nombres de paramètres différents. Les qualifications des méthodes peuvent aussi intervenir. Le choix d'une surcharge se fait à la compilation, à partir des types statiques ; le type de retour seul ne permet pas de les distinguer. La *redéfinition* fournit une implémentation dérivée d'une méthode virtuelle ; son choix dépend ensuite du type dynamique lors d'un appel virtuel.

Une déclaration dans une classe dérivée peut *masquer* les surcharges de la base portant le même nom, même avec d'autres paramètres.

```cpp
// Programme complet
#include <iostream>
struct Base {
  void afficher(int) const { std::cout << "entier\n"; }
};
struct Derivee : Base {
  using Base::afficher; // rend la surcharge de la base visible
  void afficher(const char*) const { std::cout << "texte\n"; }
};
```

```
int main() {
  Derivee d;
  d.afficher(3);        // entier ; sans using : erreur de compilation
  d.afficher("mot");    // texte
}
```

Les opérateurs peuvent aussi être surchargés. Par exemple, avec la structure `Point` et la structure `Boite` de l'exemple géométrique :

```
// Fragment ; #include <ostream>
std::ostream& operator<<(std::ostream& out, const Point& p) {
  return out << '(' << p.x << ',' << p.y << ')';
}
std::ostream& operator<<(std::ostream& out, const Boite& b) {
  return out << "[Boite bg=" << b.bg << " hd=" << b.hd << ']';
}
```

La surcharge ne change ni la priorité ni le nombre d'opérandes d'un opérateur. Elle permet notamment d'utiliser une notation mathématique pour les vecteurs ou les matrices.

2.6 Conversions et informations de type à l'exécution (RTTI)

La conversion d'un pointeur ou d'une référence vers une base publique non ambiguë est implicite : Animal& a = lion;. Dans l'autre sens, `dynamic_cast` peut vérifier à l'exécution qu'un objet appartient à une classe dérivée. Ces conversions vérifiées exigent une base polymorphe, donc possédant au moins une fonction virtuelle.

```
// A ajouter apres les definitions d'animaux
#include <iostream>
#include <typeinfo>
void examine(const Animal& a) {
  if (const auto* m = dynamic_cast<const Mammifere*>(&a))
    std::cout << m->cri() << '\n';
  else
    std::cout << "Cet animal n'est pas un mammifere\n";
  try {
    const auto& lion = dynamic_cast<const Lion&>(a);
    std::cout << "Lion : " << lion.nom() << '\n';
  } catch (const std::bad_cast&) {
    std::cout << "Ce n'est pas un lion\n";
  }
  std::cout << (typeid(a) == typeid(Lion)) << '\n';
}
int main() {
  Lion lion("Mufasa");
  Mouche mouche("Patouche");
  examine(lion);
  examine(mouche);
}
```

Un échec de `dynamic_cast<T*>` renvoie `nullptr` ; un échec de `dynamic_cast<T&>` lève `std::bad_cast`. `typeid(a)` observe ici le type dynamique car `a` désigne un objet polymorphe ; pour un type non polymorphe, il observe le type statique. Comparer à `typeid(Lion)` teste le type exact, tandis qu'un cast vers `Mammifere` accepte aussi ses classes dérivées. Le résultat de `typeid(...).name()` est dépendant de l'implémentation et ne constitue pas un nom portable pour une sauvegarde ou une interface.

Les RTTI sont activées par défaut avec GCC : il n'est pas nécessaire d'ajouter `-frtti` dans une compilation ordinaire. L'option `-fno-rtti` désactive les informations nécessaires à ces usages. `static_cast` peut effectuer certaines conversions descendantes sans vérification, mais ne valide pas le type réel : une hypothèse erronée peut conduire à un comportement indéfini. `reinterpret_cast` n'est pas un substitut à `dynamic_cast`.

Le RTTI sert ici à accéder à une capacité propre aux mammifères. Pour une opération commune à tous les animaux, préférez une méthode virtuelle à une succession de tests de type. Si les tests se multiplient, reconsidérez l'interface ou la modélisation.

2.7 Héritage multiple et héritage en diamant

Une classe peut avoir plusieurs bases, séparées par des virgules. Cela permet notamment de combiner plusieurs interfaces indépendantes. Un *diamant* apparaît lorsque deux bases ont elles-mêmes une base commune.

Considérons un appareil multifonction : il est à la fois une imprimante et un scanner, mais possède un seul identifiant d'appareil. Sans héritage virtuel, les deux branches possèdent chacune leur sous-objet `Appareil` :

```
// Programme complet ; premiere modelisation
#include <iostream>
struct Appareil {
  int identifiant;
  explicit Appareil(int id) : identifiant(id) {}
};
struct Imprimante : Appareil {
  explicit Imprimante(int id) : Appareil(id) {}
};
struct Scanner : Appareil {
  explicit Scanner(int id) : Appareil(id) {}
};
struct Multifonction : Imprimante, Scanner {
  explicit Multifonction(int id) : Imprimante(id), Scanner(id) {}
};
int main() {
  Multifonction m(42);
  // Appareil* p = &m;      // invalide : base ambigue
  // m.identifiant = 7;     // invalide : deux membres possibles
  m.Imprimante::identifiant = 7;
  std::cout << m.Scanner::identifiant << '\n'; // encore 42
}
```

Les identifiants peuvent diverger : ils ne désignent pas le même sous-objet. Avec l'héritage virtuel sur les *deux branches*, le multifonction ne contient qu'un seul sous-objet `Appareil` partagé :

```
// Programme complet ; seconde modelisation, independante de la premiere
#include <iostream>
struct Appareil {
  int identifiant;
  explicit Appareil(int id) : identifiant(id) {}
};
struct Imprimante : virtual Appareil {
  explicit Imprimante(int id) : Appareil(id) {}
  void imprimer() const { std::cout << "Impression\n"; }
};
struct Scanner : virtual Appareil {
  explicit Scanner(int id) : Appareil(id) {}
  void numeriser() const { std::cout << "Numerisation\n"; }
};
struct Multifonction : Imprimante, Scanner {
  explicit Multifonction(int id)
    : Appareil(id), Imprimante(id), Scanner(id) {}
};
int main() {
  Multifonction m(42);
  Appareil* p = &m; // conversion non ambigue
  p->identifiant = 7;
  std::cout << m.Imprimante::identifiant << ' '
            << m.Scanner::identifiant << '\n'; // 7 7
  m.imprimer();
  m.numeriser();
}
```

C'est la classe *la plus dérivée*, ici `Multifonction`, qui initialise la base virtuelle `Appareil`. Les initialiseurs `Appareil(id)` des branches ne servent que lorsque cette branche construit elle-même un objet le plus dérivé, par exemple une imprimante seule. Les bases virtuelles sont initialisées avant les bases non virtuelles.

Le mot-clé `virtual` appliqué à une base règle le partage des sous-objets ; appliqué à une méthode, il règle le dispatch des appels. L'héritage virtuel ne rend donc pas automatiquement les méthodes

virtuelles. Il ne résout pas non plus tous les conflits : si les deux branches redéfinissent une même méthode virtuelle de la base partagée, la classe finale doit fournir une redéfinition commune pour disposer d'un unique redéfinisseur final.

Cet exemple justifie le partage d'identité ; dans une application réelle, un appareil contenant des modules d'impression et de numérisation peut être plus simplement représenté par composition.

En résumé : Programmation orientée objet ; redéfinition et surcharge

- Les accès sont publics, protégés ou privés ; `static` associe un membre à la classe.
- L'héritage public exprime une relation de sous-typage ; il faut en respecter les comportements attendus.
- Une méthode virtuelle permet le dispatch dynamique ; `override` vérifie une redéfinition.
- Une classe conservant une méthode virtuelle pure est abstraite.
- Une base destinée à la destruction polymorphe doit avoir un destructeur virtuel adapté.
- Les pointeurs et références préservent le polymorphisme ; une copie par valeur peut découper l'objet.
- La surcharge est résolue à la compilation ; une méthode dérivée peut masquer celles de la base.
- `dynamic_cast` vérifie certaines conversions : pointeur nul en cas d'échec pour un pointeur, exception pour une référence.
- Dans un diamant, l'héritage virtuel partage une base, construite par la classe la plus dérivée.

Point de vue



Avant d'introduire un héritage, demandez-vous si vous représentez une relation « est un » ou « contient un ». La composition et la délégation suffisent souvent, tout en limitant les dépendances entre classes. Pour combiner des capacités, préférez lorsque c'est pertinent des interfaces légères. Le polymorphisme dynamique est utile pour des types choisis à l'exécution ; les templates offrent une autre forme de polymorphisme, résolue à la compilation.

Références techniques. Projet public du standard C++ : sections `[class.virtual]`, `[expr.dynamic.cast]`, `[expr.typeid]` et `[class.mi]`, `https://eel.is/c++draft/`. Pour les options RTTI de GCC, voir `https://gcc.gnu.org/onlinedocs/gcc/C++-Dialect-Options.html`.

3 Mécanismes d'allocation mémoire

En C++, il faut distinguer la durée de stockage, la durée de vie d'un objet et la propriété de la mémoire. Une variable locale ordinaire a une durée de stockage *automatique*, souvent réalisée sur la pile. Une variable globale ou locale `static` a une durée de stockage *statique*. Enfin, une allocation dynamique permet une durée indépendante du bloc où elle a lieu. La pile et le tas sont des modèles d'implémentation usuels, non des contraintes imposées par le langage. Le stockage propre aux threads constitue une autre catégorie, que nous ne détaillons pas ici.

Les exemples sans indication de version fonctionnent en C++11 ; seules les versions minimales postérieures sont précisées.

3.1 Objets locaux et durée de stockage automatique

Une variable locale ordinaire est construite lorsque l'exécution atteint sa définition et détruite lorsque son bloc est quitté, y compris lors de la propagation d'une exception. Les objets sont détruits

dans l'ordre inverse de leur construction ; les éléments d'un tableau le sont dans l'ordre inverse des indices. Une référence ou un pointeur n'entraîne pas, à lui seul, la construction d'un nouvel objet désigné.

```cpp
// Programme complet
#include <iostream>
struct A {
  A() { std::cout << "A::A()\n"; }
  explicit A(int) { std::cout << "A::A(int)\n"; }
  A(const A&) { std::cout << "A::A(const A&)\n"; }
  ~A() { std::cout << "A::~A()\n"; }
};
int main() {
  A a1;
  A a2(5);
  A a3(a1);
  A a4 = a1; // construction par copie, non affectation
  A as[3];
  A& ref = a1;
  A* ptr = &a2;
  (void)ref; (void)ptr;
} // sept appels au destructeur
```

Ne confondez pas ce mécanisme avec une variable locale `static` :

```cpp
// Fragment : la variable persiste entre les appels
int prochainNumero() {
  static int numero{0}; // initialise au premier passage
  return ++numero;
}
```

La durée de stockage d'un objet ne décrit pas nécessairement ses ressources : un `std::vector` local peut posséder un tableau alloué dynamiquement.

3.2 Allocation dynamique : `new`, `delete` et tableaux

Une expression `new T(...)` alloue le stockage, construit un objet, puis renvoie son adresse. `delete p` détruit l'objet puis libère son stockage. Pour les tableaux, on doit apparier `new T[n]` avec `delete[]`, même lorsque le tableau ne contient qu'un élément. La mémoire libérée n'est pas nécessairement rendue immédiatement au système d'exploitation : l'allocateur peut la conserver pour d'autres allocations.

```cpp
// Programme complet
#include <cstddef>
#include <iostream>
struct A {
  A() { std::cout << "construction\n"; }
  explicit A(int) { std::cout << "construction avec entier\n"; }
  ~A() { std::cout << "destruction\n"; }
};
A* alloueA() { return new A(5); }
A* alloueTableau(std::size_t n) { return new A[n]; }
int main() {
  A* p1 = new A;
  A* p2 = alloueA();
  A* tableau = alloueTableau(3);
  A& ref = tableau[0];
  (void)ref;
  delete p1;
  delete p2;       // indispensable : sinon fuite de memoire
  delete[] tableau;
  // p1, p2, tableau et ref ne designent plus d'objets vivants.
}
```

Le pointeur local disparaît à la fin de son bloc, mais pas l'objet alloué. Un pointeur brut ne précise pas s'il possède l'objet : il faut établir qui est responsable de sa destruction. L'exemple ci-dessus est correct sur le chemin normal ; une exception avant les `delete` pourrait laisser des allocations non libérées. Les gestionnaires de ressources présentés plus loin évitent cette fragilité.

Pour des scalaires locaux ou alloués, l'absence d'initialiseur peut laisser une valeur indéterminée. Initialisez avant toute lecture :

```cpp
// Fragment dans une fonction
int* p = new int;          // valeur non initialisee
int* q = new int{};        // zero
int* t = new int[10]{};    // dix zeros
*p = 7;                    // affecter avant de lire
// Ne jamais ecrire delete t ni delete[] q.
delete p;
delete q;
delete[] t;
```

Un `delete` sur `nullptr` ne fait rien. En revanche, supprimer deux fois le même objet, supprimer un objet local ou utiliser un objet après sa destruction provoque un comportement indéfini. Mettre un pointeur à `nullptr` après libération ne corrige pas les autres pointeurs qui référençaient le même objet.

En cas d'échec d'allocation, le `new` ordinaire lève généralement `std::bad_alloc` ; il ne renvoie pas `nullptr`. La forme `new (std::nothrow) T`, déclarée dans `<new>`, renvoie un pointeur nul en cas d'échec de l'allocation ; elle ne supprime pas une éventuelle exception du constructeur. Si un constructeur appelé par une expression `new` échoue, le stockage alloué par cette expression est libéré. Pour `new[]`, les éléments déjà construits sont également détruits.

3.3 Encapsuler un tableau : construction, destruction et propriété

Le principe RAII (*Resource Acquisition Is Initialization*) attache une ressource à la durée de vie d'un objet propriétaire. Voici une classe minimale qui alloue un tableau à sa construction et le libère dans son destructeur :

```cpp
// Programme complet
#include <cstddef>
#include <iostream>
class Tableau {
  std::size_t mTaille;
  double* mDonnees;
public:
  explicit Tableau(std::size_t n)
    : mTaille(n), mDonnees(new double[n]{}) {}
  ~Tableau() { delete[] mDonnees; }

  Tableau(const Tableau&) = delete;
  Tableau& operator=(const Tableau&) = delete;
  Tableau(Tableau&&) = delete;
  Tableau& operator=(Tableau&&) = delete;

  std::size_t size() const { return mTaille; }
  double& operator[](std::size_t i) { return mDonnees[i]; }
  const double& operator[](std::size_t i) const { return mDonnees[i]; }
};
int main() {
  Tableau t(3); // trois doubles initialises a zero
  t[1] = 2.5;
  const Tableau& vue = t;
  std::cout << vue[0] << ' ' << vue[1] << '\n'; // 0 2.5
  // Tableau copie = t; // interdit
} // delete[] automatique via ~Tableau()
```

L'accès `operator[]` suppose `i < size()`, sans vérification. Le tableau peut avoir une taille nulle ; `delete[]` reste alors valide, mais aucun élément n'est accessible.

Une copie implicite du pointeur donnerait deux propriétaires de la même zone, avec un risque de double libération : la copie est donc explicitement interdite. Le déplacement est aussi interdit ici pour garder l'exemple minimal ; une version mobile devrait transférer la propriété et remettre la source dans un état valide. C'est l'occasion d'évoquer la *règle des cinq* : lorsqu'on gère directement une ressource, il faut examiner destructeur, construction et affectation par copie, construction et affectation par déplacement.

Attention : si le corps du constructeur de `Tableau` lançait une exception après l'allocation, le destructeur de `Tableau` ne serait pas appelé, car sa construction n'aurait pas abouti. Le pointeur brut membre ne libérerait pas le tableau. Un membre propriétaire tel que `std::unique_ptr<double[]>` ou

`std::vector<double>` règle ce problème : les membres déjà construits sont détruits automatiquement. Pour un tableau redimensionnable, préférez en pratique `std::vector`.

3.4 Pointeurs intelligents et fonctions de création

L'en-tête `<memory>` fournit les pointeurs intelligents. Il faut choisir un modèle de propriété, et non seulement une syntaxe d'allocation :

Type	Rôle
`std::unique_ptr<T>`	Propriétaire exclusif, déplaçable, non copiable
`std::shared_ptr<T>`	Propriété partagée par plusieurs pointeurs
`std::weak_ptr<T>`	Observation d'une propriété partagée
`T*` ou `T&`	Accès sans propriété, par convention

Un pointeur intelligent gère la destruction ; il ne garantit ni la validité d'un indice ni la possibilité de déréférencer un pointeur vide.

3.4.1 Propriété exclusive : `unique_ptr` et `make_unique`.

`std::make_unique<T>(args...)` construit un objet `T` et renvoie un `std::unique_ptr<T>`. Les arguments sont transmis au constructeur.

```
// C++14 ; programme complet
#include <iostream>
#include <memory>
#include <string>
#include <utility>
struct Personne {
  std::string nom;
  explicit Personne(const std::string& n) : nom(n) {}
};
int main() {
  auto p = std::make_unique<Personne>("Ada");
  // Type de p : std::unique_ptr<Personne>
  std::cout << p->nom << '\n';
  // auto copie = p; // interdit : propriete exclusive
  auto q = std::move(p); // transfert, sans nouvel objet Personne
  if (!p) std::cout << "p est vide\n";
  Personne* observation = q.get(); // ne transfere pas la propriete
  std::cout << observation->nom << '\n';
  q.reset(); // detruit Personne ; observation devient invalide
}
```

Le déplacement du `unique_ptr` ne déplace pas l'objet désigné : il transfère sa propriété. Ne faites pas `delete` sur le résultat de `get()`. `release()` abandonne la propriété et renvoie le pointeur sans détruire l'objet ; son usage demande de reprendre explicitement la responsabilité de la libération.

`unique_ptr` existe en C++11, mais `make_unique` demande C++14. Un tableau s'utilise avec la spécialisation `unique_ptr<T[]>` :

```
// C++14 ; fragment ; #include <memory>
auto valeurs = std::make_unique<double[]>(10);
// Type : std::unique_ptr<double[]> ; dix valeurs initialisees a zero
valeurs[3] = 1.5;
// Liberation automatique avec delete[] ; pas de methode size().
```

La taille doit être conservée séparément. Depuis C++20, `std::make_unique_for_overwrite<double[]>(10)` permet d'éviter l'initialisation à zéro ; chaque élément doit alors être écrit avant sa lecture.

3.4.2 Propriété partagée : `shared_ptr` et `make_shared`.

`std::make_shared<T>(args...)` construit un objet et renvoie un `std::shared_ptr<T>`. Copier ce pointeur partage le même objet, sans copier `T`. L'objet est détruit lorsque le dernier propriétaire partagé le relâche.

```
// Programme complet
#include <iostream>
#include <memory>
```

```cpp
#include <string>
int main() {
  auto p = std::make_shared<std::string>("bonjour");
  // Type : std::shared_ptr<std::string>
  auto q = p;
  *q += " !";
  std::cout << *p << '\n';          // bonjour !
  std::cout << p.use_count() << '\n'; // 2 dans cet exemple
  p.reset(); // q reste proprietaire
  std::cout << *q << '\n';
} // destruction de la chaine quand q est detruit
```

La propriété partagée a un coût de gestion ; préférez `unique_ptr` lorsqu'un seul propriétaire suffit. `make_shared` permet généralement une allocation commune de l'objet et du bloc de contrôle. La gestion des compteurs ne rend pas les accès concurrents à l'objet lui-même sûrs. Ne construisez jamais deux `shared_ptr` indépendants à partir du même pointeur brut : ils pourraient détruire deux fois le même objet. Copiez le `shared_ptr` existant pour partager sa propriété.

```cpp
// C++20 ; fragment ; #include <memory>
auto valeurs = std::make_shared<double[]>(10);
// Type : std::shared_ptr<double[]> ; dix zeros
valeurs[3] = 1.5;
```

Les surcharges de `make_shared` pour les tableaux demandent C++20.

3.4.3 Observer sans prolonger la vie : `weak_ptr`.

Deux objets qui se possèdent mutuellement par des `shared_ptr` peuvent former un cycle qui empêche leur destruction. Une relation d'observation peut être représentée par un `weak_ptr`, qui ne compte pas comme propriétaire.

```cpp
// Programme complet
#include <iostream>
#include <memory>
#include <string>
int main() {
  std::weak_ptr<std::string> observation;
  {
    auto p = std::make_shared<std::string>("message");
    observation = p;
    if (auto acces = observation.lock())
      std::cout << *acces << '\n';
  } // dernier proprietaire detruit : la chaine est detruite
  if (!observation.lock())
    std::cout << "Objet disparu\n";
}
```

`lock()` renvoie un `shared_ptr` propriétaire si l'objet existe encore, sinon un pointeur partagé vide. La mémoire du bloc de contrôle peut persister jusqu'à la disparition des observateurs faibles.

Ces outils conduisent à la *règle de zéro* : si les ressources sont portées par des membres qui les gèrent eux-mêmes, la classe n'a généralement pas besoin d'écrire ses propres opérations de copie, de déplacement ou de destruction. Il faut néanmoins vérifier que leurs comportements par défaut correspondent aux invariants de la classe.

3.5 Surcharge des fonctions `operator new` et `operator delete`

Il faut distinguer l'expression `new A(...)` de la fonction d'allocation `operator new`. Cette fonction réserve le stockage ; elle ne construit pas l'objet. Le constructeur est appelé ensuite par l'expression `new`. Symétriquement, le `delete` ordinaire appelle d'abord le destructeur, puis la fonction de désallocation `operator delete`.

Une classe peut personnaliser ces fonctions, par exemple pour utiliser un pool. Voici un exemple minimal complet qui délègue aux fonctions globales :

```cpp
// Programme complet
#include <cstddef>
#include <new>
class A final {
public:
```

```
  A() = default;
  ~A() = default;
  static void* operator new(std::size_t taille) {
    return ::operator new(taille);
  }
  static void operator delete(void* p) noexcept {
    ::operator delete(p);
  }
};
int main() {
  A* p = new A; // allocation puis construction
  delete p;     // destruction puis desallocation
}
```

Le préfixe `::` évite de rappeler récursivement la fonction de la classe. Pour un pool, remplacez les appels globaux par une paire allocation/libération cohérente : l'allocateur ordinaire doit fournir un stockage assez grand et correctement aligné, ou signaler l'échec par `std::bad_alloc`. La fonction de libération ne doit pas lancer d'exception.

Si la construction échoue après l'allocation, la fonction de désallocation correspondante est appelée lorsque les règles du langage la trouvent. La surcharge de `operator new` ne personnalise pas automatiquement `operator new[]` : les tableaux et les allocations à alignement étendu nécessitent leurs propres considérations. Cet exemple ne définit pas un allocateur général pour tous ces cas. `std::make_shared` ne passe pas par le `operator new` propre à la classe ; pour contrôler son allocation, on dispose de `std::allocate_shared` avec un allocateur adapté.

En résumé : Mécanismes d'allocation mémoire

— Une variable locale ordinaire a une durée automatique ; `static` a un autre sens.
— Appariez `new`/`delete` et `new[]`/`delete[]`.
— Un pointeur brut ne libère pas automatiquement l'objet désigné.
— RAII attache la propriété d'une ressource à la durée de vie d'un objet.
— Une classe propriétaire d'un pointeur brut doit contrôler copie et déplacement.
— Préférez `unique_ptr` pour la propriété exclusive ; `shared_ptr` pour un partage nécessaire et `weak_ptr` pour l'observation associée.
— Préférez les conteneurs standard pour stocker des collections d'éléments.

Point de vue



Préférez des objets locaux et des conteneurs standard ; une taille variable ne justifie pas à elle seule une gestion manuelle de la mémoire. Si une allocation dynamique est nécessaire, exprimez immédiatement sa propriété par un conteneur ou un pointeur intelligent. Réservez les appels explicites à `new` et `delete` aux cas où vous implémentez vous-même un mécanisme de gestion de ressources. RAII simplifie la libération, mais ne dispense pas de vérifier les copies, les déplacements, les exceptions et les éventuels cycles de propriété.

Références techniques. Projet public du standard C++ : `https://eel.is/c++draft/`, sections `[expr.new]`, `[expr.delete]`, `[unique.ptr.create]`, `[util.smartptr.shared.create]` et `[class.free]`.

4 Patrons de fonctions et patrons de classes

La programmation générique consiste à exprimer un algorithme ou une structure de données en fonction des opérations nécessaires, plutôt qu'en fonction d'un type concret unique. Les *patrons*, ou *templates*, permettent de paramétrer des fonctions et des classes par des types, des valeurs connues à la compilation ou d'autres templates.

Contrairement au polymorphisme dynamique, qui choisit une redéfinition virtuelle à l'exécution, les paramètres des templates sont fixés à la compilation. Le compilateur peut alors spécialiser et optimiser le code pour ces paramètres. Cela ne signifie pas que les templates sont toujours plus rapides : disposition des données, algorithme, optimisation et taille du code interviennent aussi. Les deux formes de polymorphisme peuvent être combinées.

Les exemples sans indication de version fonctionnent en C++11 ; seules les versions minimales postérieures sont précisées.

4.1 Écriture polymorphe et écriture générique

Comparons deux versions d'un même tri à bulles. Cet algorithme quadratique est choisi pour sa brièveté ; en pratique, utilisez `std::sort` pour les conteneurs et comparateurs adaptés, avec une complexité en $O(n \log n)$ comparaisons.

Pour isoler le mécanisme d'appel, les deux versions trient le même tableau contigu de points, avec le même ordre lexicographique. On fait varier le *comparateur*, sans imposer d'héritage aux données.

```
// Definitions communes ; #include <vector> et <utility>
struct Point { double x, y; };
struct Ordre {
  virtual ~Ordre() = default;
  virtual bool avant(const Point& a, const Point& b) const = 0;
};
struct OrdreLexicographique : Ordre {
  bool avant(const Point& a, const Point& b) const override {
    return a.x < b.x || (a.x == b.x && a.y < b.y);
  }
};
```

La version polymorphe reçoit une référence vers l'interface `Ordre`. Elle accepte ainsi tout comparateur dérivé, dont le choix peut être effectué à l'exécution. Elle reste ici spécialisée sur le type des données, `Point`.

```
void triPolymorphe(std::vector<Point>& v, const Ordre& ordre) {
  for (std::size_t fin = v.size(); fin > 1; --fin)
    for (std::size_t j = 1; j < fin; ++j)
      if (ordre.avant(v[j], v[j - 1]))
        std::swap(v[j], v[j - 1]);
}
```

La version générique est paramétrée par le type des éléments et celui du comparateur. Aucun héritage n'est requis ; le comparateur doit pouvoir être appelé avec deux éléments, et les éléments doivent être échangeables.

```
template <typename T, typename Comparer>
void triGenerique(std::vector<T>& v, const Comparer& avant) {
  for (std::size_t fin = v.size(); fin > 1; --fin)
    for (std::size_t j = 1; j < fin; ++j)
      if (avant(v[j], v[j - 1]))
        std::swap(v[j], v[j - 1]);
}
struct Lexicographique {
  bool operator()(const Point& a, const Point& b) const {
    return a.x < b.x || (a.x == b.x && a.y < b.y);
  }
};
// Dans une fonction, avec deux copies des memes donnees :
// triPolymorphe(v1, OrdreLexicographique{});
// triGenerique(v2, Lexicographique{});
```

Les tableaux vides et de taille un sont pris en charge, sans calculer un pointeur avant le début du tableau. Les comparateurs doivent définir un *ordre strict faible* ; pour les points de cet exemple, on suppose des coordonnées finies, ce qui exclut notamment les valeurs NaN.

4.1.1 Mesures d'exécution.

Le tableau suivant donne des mesures réellement effectuées, en millisecondes, sur une machine virtuelle x86-64 équipée d'un Intel Xeon Platinum 8573C, avec GCC 13.3 et les options `-std=c++11 -O3`

`-DNDEBUG`, sans LTO. La fonction polymorphe est compilée dans une unité de traduction distincte qui ne voit que l'interface du comparateur, sans sa classe concrète. Les appels virtuels ont été vérifiés dans le code machine ; sinon, le compilateur pourrait les remplacer par des appels directs (*dévirtualisation*).

Chaque mesure est la médiane de sept tris après un échauffement, sur les mêmes points pseudo-aléatoires à graine fixée. L'ordre de passage des deux approches est alterné. Génération, copie, allocation, vérification du résultat et entrées/sorties sont hors chronométrage ; seul le tri est mesuré avec `std::chrono::steady_clock`.

Nombre de points	Polymorphe (ms)	Générique (ms)	Rapport
1000	1.306	0.610	2.142
2000	5.333	2.194	2.430
5000	40.194	13.126	3.062
10000	191.578	55.458	3.454
20000	870.020	474.845	1.832

Le rapport est le temps polymorphe divisé par le temps générique. Il décrit ces deux implémentations dans cet environnement, sans constituer une garantie générale : les conditions d'une machine virtuelle peuvent fluctuer. Les deux algorithmes effectuent $n(n-1)/2$ comparaisons ; l'écart observé peut notamment refléter l'appel indirect et les optimisations rendues possibles par l'inlining. Il ne mesure ni le coût d'un `dynamic_cast`, absent ici, ni celui d'une dispersion des données, identiques et contiguës dans les deux cas. Le programme reproductible est fourni avec les exemples complémentaires.

4.2 Patrons de fonctions : définition, déduction et instanciation

Un patron de fonction décrit une famille de fonctions. Le préfixe `template <typename T>` introduit un paramètre de type, utilisable dans la signature et dans le corps. Dans cette déclaration, `class T` et `typename T` sont équivalents, même si `T` est un type scalaire.

```
// Programme complet
template <typename T>
T maximum(const T& a, const T& b) {
  return a < b ? b : a;
}
int main() {
  int i = maximum(4, 17);           // T = int
  double x = maximum(3.5, 14.2);    // T = double
  double y = maximum<double>(2, 1.5); // T explicite ; conversion de 2
  (void)i; (void)x; (void)y;
  // maximum(2, 1.5); // invalide : deductions incompatibles
}
```

La *déduction* détermine les arguments du template à partir de ceux de l'appel. Elle ne cherche pas systématiquement un type commun par conversions implicites : ici `int` et `double` conduisent à deux déductions incompatibles. Un argument explicite ou une conversion comme `maximum(static_cast<double>(2), 1.5)` résout ce cas. Le type attendu pour le résultat ne détermine pas `T` dans cet exemple.

L'*instanciation* forme une spécialisation pour les arguments choisis, par exemple `maximum<int>`. Plusieurs appels avec les mêmes arguments de template utilisent la même spécialisation ; il n'y a pas une génération indépendante de code à chaque appel. Le code machine final peut ensuite être fusionné, supprimé ou intégré aux appelants par l'optimiseur. Une instanciation peut aussi être requise sans appel direct, par exemple pour prendre l'adresse de la fonction.

Le résultat est retourné *par valeur* : l'exemple copie l'opérande sélectionné et évite de renvoyer une référence vers un temporaire. Le passage des arguments par `const T&` évite leur copie à l'entrée, mais n'élimine pas celle du résultat.

4.3 Contraintes, concepts et modèles

Un template n'accepte pas « n'importe quel type » : ses opérations doivent être valides et respecter leur sens attendu. Pour `maximum`, il faut pouvoir comparer deux `const T&`, interpréter le résultat comme une condition et copier la valeur choisie. L'opérateur `<` peut être intégré au langage, défini comme membre ou comme fonction libre ; il n'est pas nécessaire que `T` possède une méthode `operator<`.

Un *concept*, au sens de la programmation générique, rassemble ces exigences syntaxiques et sémantiques. Un type qui les respecte est un *modèle* du concept. Par exemple, les entiers usuels permettent la copie et une comparaison ordonnée ; une classe qui ne fournit aucun ordre ne convient pas, même si elle est copiable.

```
// Fragment utilisant maximum
#include <string>
struct Chou { std::string nom; };
// maximum(Chou{"rouge"}, Chou{"de Bruxelles"}); // pas de operator<
```

Sans contrainte explicite, l'erreur peut apparaître lors de l'instanciation du corps du template, avec un diagnostic parfois difficile à lire.

4.3.1 Première formulation avec les concepts du langage (C++20).

Le mot-clé `concept` permet de nommer une contrainte et `requires` d'exprimer les opérations attendues. Voici une version contrainte de `maximum`, indépendante de la précédente :

```
// C++20 ; programme complet
#include <concepts>
template <typename T>
concept ComparableCopiable = std::copy_constructible<T>
  && requires(const T& a, const T& b) {
     { a < b } -> std::convertible_to<bool>;
   };
template <ComparableCopiable T>
T maximum(const T& a, const T& b) {
  return a < b ? b : a;
}
int main() { return maximum(2, 3) == 3 ? 0 : 1; }
```

`template <ComparableCopiable T>` peut aussi s'écrire `template <typename T> requires` ComparableCopiable `<T>`. Les concepts employés ici imposent des exigences légèrement plus fortes que le corps seul, notamment la convertibilité implicite de la comparaison en `bool`. Ils vérifient des propriétés syntaxiques ; ils ne prouvent pas, à eux seuls, que l'ordre est transitif ou cohérent. Les exigences sémantiques restent à documenter et à respecter. Les concepts standard et leur emploi algorithmique seront approfondis dans la partie sur la généricité.

4.4 Déclarations, définitions et organisation des fichiers

La définition d'un template doit généralement être accessible lorsqu'une spécialisation est implicitement instanciée. La pratique habituelle consiste à la mettre dans un en-tête, protégé contre les inclusions multiples, ou dans un fichier inclus par cet en-tête :

```
// maximum.hpp
#ifndef COURS_MAXIMUM_HPP
#define COURS_MAXIMUM_HPP
template <typename T>
T maximum(const T& a, const T& b);
#include "maximum.tpp"
#endif
```

```
// maximum.tpp : pas de point-virgule apres la signature !
template <typename T>
T maximum(const T& a, const T& b) {
  return a < b ? b : a;
}
```

Un fichier `.tpp` n'est pas compilé séparément : son inclusion rend la définition accessible aux unités de traduction utilisatrices. Plusieurs instanciations identiques dans des unités différentes sont autorisées sous les règles du langage, notamment celles de définition unique.

L'*instanciation explicite* permet une autre organisation lorsqu'on fixe les spécialisations disponibles : après la définition dans un fichier source, `template int` maximum`<int>(const int&, const int&);` demande l'instanciation de ce cas. Cette technique ne rend pas automatiquement disponibles les autres types. C++20 introduit aussi les modules ; `export` sert alors à exposer des déclarations d'un module, et ne rétablit pas l'ancien mécanisme d'exportation séparée des templates, retiré en C++11.

4.5 Plusieurs paramètres et objets appelables

Un algorithme peut être paramétré par plusieurs types indépendants. Dans cet exemple, le premier représente un itérateur et le second un objet appelable :

```
// Programme complet
#include <iostream>
template <typename Iterateur, typename Fonction>
void pour_chacun_faire(Iterateur debut, Iterateur fin, Fonction& f) {
  for (; debut != fin; ++debut) f(*debut);
}
struct Accumulateur {
  double cumul{0.0};
  void operator()(double valeur) { cumul += valeur; }
};
int main() {
  double valeurs[] = {4.5, 12.3, -2.5, 8.4, 9.8};
  Accumulateur acc;
  pour_chacun_faire(valeurs, valeurs + 5, acc);
  std::cout << acc.cumul << '\n'; // 32.5
}
```

L'intervalle est semi-ouvert, `[debut, fin)` : `fin` n'est jamais déréférencé. Il doit être possible d'atteindre `fin` par incrémentations valides. L'itérateur doit supporter les expressions utilisées, et `f(*debut)` doit être valide ; le paramètre de `f` n'a pas besoin d'avoir exactement le type de l'élément si une conversion appropriée existe. Un appelable peut être un foncteur, une lambda ou une fonction :

```
// Fragment dans une fonction, utilisant pour_chacun_faire
int valeurs[] = {-2, 0, 3, 5};
int positifs = 0;
auto compte = [&positifs](int x) { if (x > 0) ++positifs; };
pour_chacun_faire(valeurs, valeurs + 4, compte); // positifs vaut 2
```

Le passage par référence conserve ici les modifications du foncteur. `std::for_each`, dans `<algorithm>`, propose un service analogue, mais prend son appelable par valeur et le retourne.

4.6 Patrons de classes : types et paramètres de valeur

Un patron de classe décrit une famille de types. Des spécialisations telles que `Paire<int, double>` et `Paire<double, int>` sont des types distincts, sans relation de sous-typage automatique.

```
// Programme complet
#include <string>
template <typename First, typename Second = First>
struct Paire {
  First premier;
  Second second;
};
int main() {
  Paire<int, std::string> p{3, "trois"};
  Paire<int> q{1, 2}; // Second = int par defaut
  (void)p; (void)q;
}
```

Les opérations spéciales dépendent des membres : une paire n'est copiable que si ses deux membres le sont. Une classe template ne doit pas forcément être copiable ou constructible par défaut pour exister. Les définitions des fonctions membres non utilisées ne sont généralement pas instanciées ; les exigences peuvent donc dépendre de l'opération effectivement appelée. Pour ce service, la bibliothèque fournit `std::pair` dans `<utility>`.

Un paramètre de valeur est lui aussi fixé à la compilation. Voici un tableau à taille fixe, fondé sur `std::array` :

```
// Definitions de FArray ; #include <array>, <cassert>, <cstddef>
template <typename T, std::size_t N>
class FArray {
  std::array<T, N> donnees{};
public:
  T& operator[](std::size_t i) {
    assert(i < N);
    return donnees[i];
```

```
  }
  const T& operator[](std::size_t i) const {
    assert(i < N);
    return donnees[i];
  }
  constexpr std::size_t size() const { return N; }
};
```

`N` fait partie du type, contrairement à la taille d'un `std::vector`. Ce paramètre n'est pas modifiable à l'exécution. L'initialiseur `donnees{}` initialise les éléments, notamment à zéro pour les types arithmétiques. `assert` est désactivé si `NDEBUG` est défini : il ne remplace pas un contrôle de bornes permanent.

```
// A ajouter apres les definitions de FArray
#include <cmath>
#include <iostream>
int main() {
  using Ligne = FArray<double, 3>;
  using Matrice = FArray<Ligne, 3>;
  Matrice m; // tous les coefficients sont initialises a zero
  const double angle = std::acos(-1.0) / 4;
  m[0][0] = std::cos(angle); m[0][1] = std::sin(angle);
  m[1][0] = -std::sin(angle); m[1][1] = std::cos(angle);
  m[2][2] = 1;
  const Matrice copie = m;
  for (std::size_t i = 0; i < 3; ++i) {
    for (std::size_t j = 0; j < 3; ++j)
      std::cout << copie[i][j] << ' ';
    std::cout << '\n';
  }
}
```

`M_PI` n'étant pas garanti par le standard, on utilise ici `std::acos(-1.0)`. Depuis C++20, `std::numbers::pi`, dans `<numbers>`, fournit directement cette constante.

Les paramètres de valeur ne se limitent pas aux entiers et aux booléens : les pointeurs, références et certains autres types sont également possibles sous les règles du standard. C++17 permet notamment `template <auto N>`, qui déduit le type de la valeur, et C++20 autorise des types structurels supplémentaires, dont certains types de classes.

4.6.1 Déduction des arguments d'une classe (C++17).

C++17 permet dans certains cas de déduire les arguments d'une classe template à partir de l'initialisation, mécanisme appelé CTAD :

```
// C++17 ; fragment ; #include <utility>
std::pair p{3, 2.5}; // std::pair<int, double>
```

Cette déduction dépend des constructeurs et des guides de déduction disponibles ; elle ne fonctionne pas arbitrairement pour toute classe template. Les règles pour les agrégats ont aussi été enrichies en C++20.

4.7 Spécialisation des patrons de classes

Une *spécialisation partielle* remplace la définition générale pour une famille de paramètres ; une *spécialisation explicite complète* fixe tous les paramètres. La spécialisation fournit sa propre définition : elle n'hérite pas automatiquement des membres du patron principal.

Par exemple, la spécialisation suivante stocke directement l'unique élément :

```
// Apres le patron principal de FArray
template <typename T>
class FArray<T, 1> {
  T valeur{};
public:
  FArray() = default;
  explicit FArray(const T& v) : valeur(v) {}
  T& operator[](std::size_t i) { assert(i == 0); return valeur; }
  const T& operator[](std::size_t i) const {
    assert(i == 0); return valeur;
```

```
  }
  constexpr std::size_t size() const { return 1; }
};
```

Le constructeur explicite évite une conversion implicite d'un `T` vers un tableau ; nous n'ajoutons pas de conversion implicite inverse. Ce cas est surtout illustratif : `std::array<T, 1>` est déjà très compact.

Pour les booléens, on peut regrouper les valeurs dans une représentation en bits, en déléguant sa gestion à `std::bitset` :

```
// #include <bitset> ; apres le patron principal
template <std::size_t N>
class FArray<bool, N> {
  std::bitset<N> bits{};
public:
  using Reference = typename std::bitset<N>::reference;
  Reference operator[](std::size_t i) {
    assert(i < N); return bits[i];
  }
  bool operator[](std::size_t i) const {
    assert(i < N); return bits[i];
  }
  constexpr std::size_t size() const { return N; }
};
```

Le retour modifiable n'est pas un `bool&` : un bit n'est pas un objet `bool` adressable. Le *proxy* renvoyé permet néanmoins `bits[3] = true` et une conversion en `bool`. La représentation compacte peut économiser de la place pour les grands tableaux, sans garantir une taille exacte ni une accélération pour les petits. Le mot-clé `typename` indique ici que le nom dépendant `std::bitset<N>::reference` désigne un type.

Si ces deux spécialisations partielles sont présentes, `FArray<bool, 1>` correspond aux deux, sans qu'une soit plus spécialisée que l'autre. Une spécialisation complète lève l'ambiguïté :

```
template <>
class FArray<bool, 1> {
  bool valeur{false};
public:
  bool& operator[](std::size_t i) { assert(i == 0); return valeur; }
  bool operator[](std::size_t i) const {
    assert(i == 0); return valeur;
  }
  constexpr std::size_t size() const { return 1; }
};
```

Déclarez les spécialisations avant les utilisations qui nécessitent leur instanciation. Une spécialisation doit préserver les propriétés de l'interface sur lesquelles les algorithmes utilisateurs s'appuient ; un retour par proxy, par exemple, change certaines possibilités d'utilisation.

4.8 Surcharge et spécialisation des patrons de fonctions

Une fonction template peut être complètement spécialisée, mais pas partiellement spécialisée. Une surcharge constitue souvent une solution plus lisible :

```
// Programme complet
template <typename T>
T maximum(const T& a, const T& b) { return a < b ? b : a; }
bool maximum(bool a, bool b) { return a || b; }
int main() {
  bool a = maximum(false, true);        // surcharge non template
  bool b = maximum<bool>(false, true); // template demande explicitement
  return a && b ? 0 : 1;
}
```

La surcharge est choisie par la résolution des surcharges ; une spécialisation explicite ne constitue pas une surcharge indépendante. Pour des spécialisations partielles, on peut déléguer l'implémentation à une classe template auxiliaire. Il faut distinguer ces mécanismes de l'instanciation, qui produit un cas à partir d'une définition existante.

4.9 Nombre variable de paramètres : templates variadiques

Un *pack* représente un nombre variable de paramètres. La notation `typename... Suite` déclare un pack de types ; `suite...` développe les arguments correspondants. L'exemple suivant conserve une formulation récursive et impose des types identiques :

```
// Programme complet
#include <iostream>
#include <type_traits>
template <typename T>
T maximum(const T& x) { return x; } // surcharge terminale

template <typename T, typename U, typename... Suite>
T maximum(const T& x, const U& y, const Suite&... suite) {
  static_assert(std::is_same<T, U>::value, "Types identiques requis");
  T reste = maximum(y, suite...);
  return x < reste ? reste : x;
}
int main() {
  std::cout << maximum(3, 7, 5, 2, 6) << '\n'; // 7
  // maximum(3, 7.1); // rejete par static_assert
}
```

Le cas à un argument est une *surcharge*, non une spécialisation. Chaque étape compare les types voisins ; une différence est rejetée à l'instanciation. Le nombre d'arguments doit être au moins un. `sizeof...(Suite)` donne la taille d'un pack.

C++17 permet des *expressions de repli* (*fold expressions*) qui évitent parfois la récursion explicite :

```
// C++17 ; programme complet
template <typename... T>
auto somme(T... valeurs) {
  static_assert(sizeof...(T) > 0, "Au moins un argument");
  return (... + valeurs); // repli a gauche
}
int main() { return somme(1, 2, 3) == 6 ? 0 : 1; }
```

Les additions intermédiaires doivent être valides ; les arguments ne doivent pas nécessairement avoir tous le même type. L'associativité choisie compte notamment pour les types numériques et les opérateurs surchargés.

En résumé : Patrons de fonctions et patrons de classes

- Un template décrit une famille paramétrée ; ses arguments sont fixés à la compilation.
- Déduction, instanciation et spécialisation sont des mécanismes distincts.
- Les exigences portent sur les opérations utilisées et leur sémantique, pas nécessairement sur une hiérarchie de classes.
- C++20 permet d'exprimer des contraintes avec `concept` et `requires`.
- Les définitions sont généralement placées dans des en-têtes ou des fichiers inclus par ceux-ci.
- Une classe template peut être spécialisée partiellement ou complètement ; une fonction template ne peut pas l'être partiellement.
- Les gains de performance dépendent du cas ; ils doivent être mesurés avec un protocole explicite.

Point de vue



Identifier les opérations réellement nécessaires à un algorithme est essentiel pour construire une interface générique utile. Évitez les contraintes inutiles, mais documentez les exigences sémantiques que le compilateur ne peut pas prouver. Choisissez le polymorphisme dynamique pour une variation de comportement à l'exécution et les templates lorsque les paramètres sont connus à la compilation, sans opposer systématiquement les deux approches.

Références techniques. Projet public du standard C++ : `https://eel.is/c++draft/`, sections sur la déduction (`[temp.deduct]`), l'instanciation (`[temp.inst]`), les spécialisations partielles (`[temp.spec.partial]`) et les concepts (`[temp.concept]`).

5 Généricité et bibliothèque standard : conteneurs, itérateurs et algorithmes

La bibliothèque standard C++ comprend des conteneurs, des itérateurs, des algorithmes, des objets appelables et, depuis C++20, des *ranges*. La STL désigne historiquement l'ensemble de ces outils génériques ; elle fait partie de la bibliothèque standard, qui comprend aussi les flux, les chaînes, les nombres aléatoires, le système de fichiers, etc.

Un conteneur stocke des éléments d'un type donné et définit ses opérations. Un algorithme générique travaille sur un intervalle et s'appuie sur les capacités de ses itérateurs, indépendamment du conteneur précis. Les exemples sans indication fonctionnent en C++11 ; seules les versions minimales postérieures sont mentionnées. Les programmes complets contiennent leurs en-têtes ; les fragments le précisent séparément.

5.1 Choisir un conteneur

On choisit une structure en fonction de l'accès, des insertions, de la recherche, de la stabilité des références et de la disposition des éléments. Les complexités asymptotiques ne suffisent pas : allocations, localité mémoire, coût des comparaisons et des déplacements influencent aussi les performances.

5.1.1 Conteneurs séquentiels

- `std::vector<T>` : tableau dynamique contigu, sauf spécialisation `vector<bool>`. Accès par indice en $O(1)$, insertion en fin en temps amorti $O(1)$; une insertion ou suppression au milieu déplace les éléments suivants et coûte linéairement en leur nombre.
- `std::array<T, N>` : tableau contigu de taille fixe `N`, qui fait partie du type. Sa taille ne peut pas être modifiée.
- `std::deque<T>` : séquence à accès aléatoire, avec insertions et suppressions efficaces aux deux extrémités ; ses éléments ne sont pas nécessairement contigus.
- `std::list<T>` : liste doublement chaînée. Insertion et suppression d'un élément en $O(1)$ à une position déjà connue, mais recherche d'une position en temps linéaire et pas d'accès par indice en $O(1)$.
- `std::forward_list<T>` : liste simplement chaînée, parcourable vers l'avant ; certaines opérations se font après un itérateur donné.

Le dernier élément d'une `list` est accessible en $O(1)$ par `back()`. Le coût exact en mémoire des conteneurs dépend des types, des allocateurs et de l'implémentation : un `vector` peut par exemple réserver plus de place que sa taille, tandis qu'une liste contient des liens et généralement plusieurs allocations.

```
// Programme complet
#include <iostream>
#include <vector>
int main() {
  std::vector<int> valeurs{1, 2, 3};
  valeurs.push_back(4);
  valeurs.reserve(20);  // reserve une capacite, pas des elements
  std::cout << valeurs.size() << '\n'; // 4
  for (int x : valeurs) std::cout << x << ' ';
  std::cout << '\n';
}
```

`reserve(n)` ne change pas la taille, alors que `resize(n)` la modifie. Un `push_back` qui entraîne une réallocation invalide tous les itérateurs, pointeurs et références sur les éléments du vecteur ; sans réallocation, il invalide l'ancien itérateur `end()`. Les autres opérations possèdent leurs propres règles d'invalidation : consultez-les avant de conserver un itérateur à travers une modification.

5.1.2 Conteneurs associatifs

`std::set` et `std::map` conservent respectivement des clés uniques et des paires clé–valeur ordonnées. `multiset` et `multimap` autorisent plusieurs éléments à clés équivalentes. L'équivalence pour le comparateur `comp` est définie par !comp(a,b)&& !comp(b,a) : elle n'est pas nécessairement l'égalité `a == b`. Le comparateur doit définir un ordre strict faible. La recherche et l'insertion sont logarithmiques dans les conteneurs ordonnés.

`std::unordered_set`, `unordered_map` et leurs versions `multi` utilisent un hachage : recherche et insertion sont en moyenne $O(1)$, mais leur pire cas peut être linéaire. Leur ordre de parcours n'est pas un ordre de tri. L'égalité et le hachage doivent être cohérents : deux clés considérées égales doivent produire le même hachage. Un *rehash* invalide les itérateurs ; les références et pointeurs vers les éléments demeurent valides, sauf suppression des éléments concernés.

```cpp
// Programme complet
#include <iostream>
#include <map>
#include <string>
int main() {
  std::map<std::string, std::string> annuaire{
    {"Lebaudu", "0479624545"}, {"Grospalais", "0479633678"},
    {"Encornet", "0479676543"}, {"Estraminot", "0475678765"}
  };
  auto it = annuaire.find("Grospalais");
  if (it != annuaire.end()) std::cout << it->second << '\n';
  // Les cles dans ["D", "L") ; L lui-meme est exclu.
  for (auto p = annuaire.lower_bound("D");
       p != annuaire.lower_bound("L"); ++p)
    std::cout << p->first << " : " << p->second << '\n';
}
```

Pour un conteneur ordonné, préférez sa méthode `lower_bound` à l'algorithme générique du même nom : les comparaisons sont logarithmiques dans les deux cas, mais la version générique peut effectuer un nombre linéaire d'avancées avec des itérateurs bidirectionnels. L'expression annuaire[cle] insère une valeur initialisée par défaut si la clé est absente ; pour une recherche sans insertion, utilisez `find`, `at` ou, depuis C++20, `contains`.

5.1.3 Adaptateurs et autres structures

`std::stack`, `std::queue` et `std::priority_queue` exposent respectivement une pile, une file et une file de priorité. Ce sont des *adaptateurs de conteneurs* : ils ne fournissent pas d'itérateurs publics. Par défaut, `stack` et `queue` utilisent un `deque`, et `priority_queue` utilise un `vector`. On peut choisir un conteneur sous-jacent compatible. `priority_queue` expose par défaut le plus grand élément ; insérer et retirer coûtent logarithmiquement.

`std::bitset<N>` contient `N` bits. La spécialisation `std::vector<bool>` peut être compacte : ses accès modifiables peuvent renvoyer des objets mandataires (*proxies*), pas de vraies références `bool&`. Depuis C++17, `std::optional<T>` représente une valeur éventuellement absente ; il ne s'agit pas d'un conteneur général.

C++23 ajoute `std::flat_set`, `flat_multiset`, `flat_map` et `flat_multimap` : adaptateurs associatifs ordonnés reposant sur des séquences sous-jacentes. Leur disposition peut favoriser le parcours, au prix d'insertions ou suppressions potentiellement linéaires. Leur disponibilité dépend de la bibliothèque standard utilisée.

5.1.4 Vue sur une séquence contiguë : `span` (C++20)

`std::span<T>` représente une suite contiguë sans posséder ses éléments. Il permet de passer un tableau ou un vecteur à une fonction sans copie :

```cpp
// C++20 ; programme complet
#include <array>
#include <span>
#include <vector>
int somme(std::span<const int> valeurs) {
  int resultat = 0;
  for (int x : valeurs) resultat += x;
```

```
  return resultat;
}
int main() {
  std::vector<int> v{1, 2, 3};
  std::array<int, 2> a{{4, 5}};
  return (somme(v) == 6 && somme(a) == 9) ? 0 : 1;
}
```

Le conteneur doit rester vivant pendant l'utilisation de la vue. Une réallocation du vecteur invalide le `span` qui le désigne. Une `std::string_view` (C++17) joue un rôle voisin pour une séquence de caractères. Ces vues ne redimensionnent pas et ne prolongent pas la durée de vie des données qu'elles observent.

5.2 Itérateurs et intervalles

Un itérateur généralise certaines opérations des pointeurs : `*it` accède à un élément et `++it` avance. Un intervalle `[debut, fin)` contient `debut` et exclut `fin` ; `fin` n'est pas déréférençable. Un intervalle vide vérifie `debut == fin`. Les deux bornes doivent former un intervalle valide.

```
// Programme complet
#include <iostream>
#include <vector>
int main() {
  std::vector<int> v{1, 2, 3, 4, 5};
  for (auto it = v.cbegin(); it != v.cend(); ++it)
    std::cout << *it << ' ';
  std::cout << '\n';
}
```

`begin/end`, `cbegin/cend` et, lorsqu'ils sont disponibles, `rbegin/rend` fournissent les bornes du parcours ordinaire, constant ou inverse. Un `const_iterator` interdit la modification de l'élément par son intermédiaire. Une boucle `for (const auto& x : v)` effectue un parcours en lecture ; `for (auto& x : v)` permet généralement la modification. Les types à proxy comme `vector<bool>` demandent une attention particulière.

5.2.1 Catégories classiques et concepts C++20

Les algorithmes classiques documentent leurs exigences en termes de catégories d'itérateurs. C++20 fournit aussi des concepts formels dans `<iterator>`. Du plus général au plus riche, côté lecture :

- `std::input_iterator` : lecture et progression, éventuellement en un seul passage ; deux copies ne garantissent pas des parcours indépendants.
- `std::forward_iterator` : parcours vers l'avant et garantie de multipassage.
- `std::bidirectional_iterator` : ajoute le recul avec `-`.
- `std::random_access_iterator` : déplacements par une distance, différence et accès par indice, en temps constant.
- `std::contiguous_iterator` : accès aléatoire avec une relation de contiguïté entre les adresses des éléments.

`std::output_iterator<I, T>` décrit l'écriture de valeurs de type `T` : il ne s'insère pas comme une étape dans cette hiérarchie de lecture. Un même itérateur peut satisfaire des exigences de lecture et d'écriture. Les concepts précisent aussi des propriétés sémantiques et de complexité que le compilateur ne peut pas toujours vérifier.

Les itérateurs de `forward_list` sont vers l'avant, ceux de `list`, `set` et `map` sont bidirectionnels ; ceux de `deque` sont à accès aléatoire mais pas nécessairement contigus. Ceux de `array` et de `vector<T>` pour `T != bool` sont contigus. Les catégories historiques et les concepts C++20 ne sont pas des règles interchangeables à l'identique, en particulier pour les proxies. `std::sort` et `std::ranges::sort` demandent l'accès aléatoire ; une `list` se trie avec sa méthode `sort()`.

5.2.2 Sentinelles, types associés et contraintes

Dans les algorithmes classiques, les deux bornes ont souvent le même type. En C++20, une *sentinelle* peut avoir un type différent de celui de l'itérateur ; `std::sentinel_for<S, I>` décrit leur

compatibilité. Le programme suivant accepte ainsi un itérateur d'entrée et une sentinelle, et exprime la contrainte du prédicat. Pour le flux, leurs types sont distincts :

```cpp
// C++20 ; programme complet
#include <functional>
#include <iterator>
#include <ranges>
#include <sstream>
#include <vector>

// C++20
#include <cstddef>
#include <iterator>

template <std::input_iterator I, std::sentinel_for<I> S,
          typename Predicat>
std::size_t compte_si(I debut, S fin, Predicat predicat)
{
    std::size_t total = 0;
    for (; debut != fin; ++debut)
        if (predicat(*debut))
            ++total;
    return total;
}
int main() {
  std::vector<int> v{-2, 0, 3, 5};
  auto positifs = [](int x) { return x > 0; };
  auto n = compte_si( v.begin(), v.end(), positifs ); // 2
  std::istringstream flux("-2 0 3");
  auto bornes = std::ranges::istream_view<int>( flux );
  // bornes.end() est ici de type std::default_sentinel_t
  auto m = compte_si( bornes.begin(), bornes.end(), positifs ); // 1
  return (n == 2 && m == 1) ? 0 : 1;
}
```

Le type de valeur associé à un itérateur classique est accessible par `std::iterator_traits<I>::value_type`. C++20 ajoute `std::iter_value_t<I>`, `std::iter_reference_t<I>` et `std::iter_difference_t<I>`. Le type de `*it` n'est pas forcément `T&` : il peut être une référence constante ou un proxy. `std::distance` calcule une distance, `std::advance` déplace un itérateur et `std::next` renvoie une copie déplacée ; leur coût dépend de la catégorie.

5.3 Algorithmes de la bibliothèque standard

La plupart des algorithmes suivants sont déclarés dans `<algorithm>`. Ils ne vérifient pas systématiquement leurs préconditions : intervalle valide, capacité de sortie suffisante, comparaison cohérente, données triées lorsque nécessaire, etc. Le nom et la catégorie ne suffisent pas à connaître toutes les exigences : consultez la signature et la documentation.

Voici un aide-mémoire des opérations classiques. Sauf mention contraire, les noms ci-dessous désignent les versions `std::` ; les variantes `std::ranges::` disponibles depuis C++20 ont parfois des signatures et des types de retour différents.

5.3.1 Parcours et recherche (sans modification)

- `all_of`, `any_of`, `none_of` : Vérifier qu'une propriété est vraie pour tous, au moins un, ou aucun élément.
- `for_each` : Appliquer une fonction à chaque élément.
- `find`, `find_if`, `find_if_not` : Trouver la première valeur égale, vérifiant un prédicat ou ne le vérifiant pas.
- `find_end`, `find_first_of` : Trouver la dernière occurrence d'une sous-séquence ou le premier élément appartenant à une autre séquence.
- `adjacent_find` : Trouver la première paire d'éléments voisins égaux ou vérifiant un prédicat.
- `count`, `count_if` : Compter les occurrences d'une valeur ou les éléments vérifiant un prédicat.

— `mismatch`, `equal`, `is_permutation` : Comparer deux intervalles : première différence, égalité ou permutation.
— `search`, `search_n` : Trouver une sous-séquence ou un nombre donné de valeurs consécutives.

5.3.2 Modification et copie de séquences

— `copy`, `copy_n`, `copy_if`, `copy_backward` : Copier un intervalle, un nombre d'éléments, une sélection ou en allant de droite à gauche.
— `move`, `move_backward` : Transférer les valeurs par affectation de déplacement, vers l'avant ou vers l'arrière.
— `swap`, `swap_ranges`, `iter_swap` : Échanger deux objets, deux intervalles ou les valeurs désignées par deux itérateurs.
— `transform` : Appliquer une fonction unaire ou binaire et écrire les résultats.
— `replace`, `replace_if`, `replace_copy`, `replace_copy_if` : Remplacer des valeurs en place ou au cours d'une copie, selon une valeur ou un prédicat.
— `fill`, `fill_n`, `generate`, `generate_n` : Remplir par une valeur ou un générateur sur un intervalle ou un nombre donné d'éléments.
— `remove`, `remove_if` : Déplacer les éléments conservés au début et renvoyer la nouvelle fin logique, sans réduire le conteneur.
— `remove_copy`, `remove_copy_if` : Copier les éléments conservés selon une valeur ou un prédicat.
— `unique`, `unique_copy` : Compacter les doublons consécutifs en place ou au cours d'une copie.
— `reverse`, `reverse_copy` : Inverser l'ordre des éléments en place ou en copiant.
— `rotate`, `rotate_copy` : Faire une rotation de l'intervalle en place ou en copiant.
— `shuffle` : Mélanger les éléments avec un générateur aléatoire fourni.
— `random_shuffle` : Ancien mélange : déprécié en C++14 et supprimé en C++17 ; utiliser shuffle.

5.3.3 Partitionnement et tri

— `is_partitioned`, `partition`, `stable_partition` : Tester ou répartir selon un prédicat ; stable_partition préserve l'ordre relatif dans chaque groupe.
— `partition_copy`, `partition_point` : Copier les deux groupes dans deux sorties ou trouver la séparation d'un intervalle déjà partitionné.
— `sort`, `stable_sort` : Trier ; stable_sort préserve l'ordre relatif des éléments équivalents.
— `partial_sort`, `partial_sort_copy` : Trier les premiers éléments sélectionnés, en place ou dans une sortie.
— `is_sorted`, `is_sorted_until` : Tester le tri ou renvoyer la fin du plus grand préfixe trié.
— `nth_element` : Placer l'élément de rang donné comme dans un tri, en partitionnant autour de lui ; les deux parties ne sont pas nécessairement triées.

5.3.4 Recherche dichotomique et opérations sur séquences triées

— `lower_bound`, `upper_bound`, `equal_range`, `binary_search` : Trouver les bornes d'une valeur, l'intervalle des éléments équivalents ou tester sa présence ; les préconditions de partitionnement doivent être satisfaites.
— `merge`, `inplace_merge` : Fusionner deux intervalles triés, dans une sortie ou lorsqu'ils sont consécutifs.
— `includes` : Tester si une séquence triée contient une autre en tenant compte des multiplicités.
— `set_union`, `set_intersection`, `set_difference`, `set_symmetric_difference` : Calculer les opérations ensemblistes sur des séquences triées, en tenant compte des multiplicités.

5.3.5 Tas, extrema et permutations

— `make_heap`, `push_heap`, `pop_heap`, `sort_heap` : Construire un tas, intégrer le dernier élément, placer la racine en fin, ou trier un tas.
— `is_heap`, `is_heap_until` : Tester la propriété de tas ou trouver la fin du préfixe qui la satisfait.
— `min`, `max`, `minmax` : Comparer deux valeurs ou un ensemble passé comme liste initialisatrice.

— `min_element`, `max_element`, `minmax_element` : Trouver les positions des extrema d'un intervalle.
— `lexicographical_compare` : Comparer deux séquences dans l'ordre lexicographique.
— `next_permutation` : obtenir la permutation lexicographique suivante.
— `prev_permutation` : obtenir la permutation lexicographique précédente.

5.3.6 Algorithmes numériques et ajouts récents

L'en-tête `<numeric>` complète `<algorithm>` :
— `accumulate` : accumulation séquentielle dans l'ordre du parcours.
— `inner_product` : produit scalaire généralisé de deux séquences.
— `partial_sum`, `adjacent_difference` : sommes cumulées et différences entre éléments voisins.
— `iota` : remplir par une suite de valeurs successives.
— `reduce`, `transform_reduce`, `inclusive_scan`, `exclusive_scan` : réductions et sommes préfixes généralisées (C++17). `reduce` peut modifier l'ordre et le regroupement des opérations ; avec des flottants, son résultat peut donc différer de `accumulate`.

C++17 ajoute par exemple `clamp` pour borner une valeur et `sample` pour échantillonner. C++20 introduit les algorithmes `std::ranges::` et `std::erase`/`std::erase_if` pour divers conteneurs. C++23 ajoute notamment `std::ranges::contains`, `starts_with` et `ends_with` pour les ranges. Les versions standard et la prise en charge effective de la bibliothèque sont à vérifier avant leur utilisation.

5.4 Exemples d'algorithmes

5.4.1 Trier et éliminer les doublons

```cpp
// Programme complet
#include <algorithm>
#include <iostream>
#include <vector>
int main() {
  std::vector<int> v{5, 2, 7, 12, 9, 7, 4, 3, 2, 15, 4, 6};
  std::sort(v.begin(), v.end());
  auto nouvelleFin = std::unique(v.begin(), v.end());
  v.erase(nouvelleFin, v.end());
  for (int x : v) std::cout << x << ' ';
  std::cout << '\n'; // 2 3 4 5 6 7 9 12 15
}
```

`unique` ne supprime que les doublons *consécutifs*. Il compacte les éléments conservés et renvoie une nouvelle fin logique ; la taille du vecteur ne change qu'avec `erase`. Le tri rend ici tous les doublons adjacents. Cette méthode modifie l'ordre initial ; un `set` ou une stratégie de hachage offrent d'autres choix.

La même distinction s'applique à `remove` et `remove_if` :

```cpp
// Programme complet
#include <algorithm>
#include <vector>
int main() {
  std::vector<int> v{1, 2, 3, 2, 4};
  v.erase(std::remove(v.begin(), v.end(), 2), v.end());
  return v == std::vector<int>{1, 3, 4} ? 0 : 1;
}
```

Depuis C++20, std::erase(v, 2) et std::erase_if(v, predicat) combinent l'opération et l'effacement pour les conteneurs qui proposent ces surcharges ; elles renvoient le nombre d'éléments effacés. `std::list` possède aussi ses propres méthodes `remove`, `remove_if` et `unique`.

5.4.2 Sélectionner les entrées d'un annuaire

L'exemple suivant utilise `copy_if` et un itérateur d'insertion. Il garde les noms compris entre `"D"` inclus et `"L"` exclu :

```cpp
// Programme complet
#include <algorithm>
```

```cpp
#include <iostream>
#include <iterator>
#include <map>
#include <string>
#include <utility>
#include <vector>
int main() {
  std::map<std::string, std::string> annuaire{
    {"Lebaudu", "0479624545"}, {"Grospalais", "0479633678"},
    {"Encornet", "0479676543"}, {"Estraminot", "0475678765"},
    {"Youplein", "0479898312"}, {"Amovibel", "0479812112"},
    {"Marmiton", "0479767842"}
  };
  std::vector<std::pair<std::string, std::string>> selection;
  std::copy_if(annuaire.begin(), annuaire.end(),
               std::back_inserter(selection),
               [](const std::pair<const std::string, std::string>& e) {
                 return e.first >= "D" && e.first < "L";
               });
  for (const auto& e : selection)
    std::cout << e.first << " : " << e.second << '\n';
}
```

`back_inserter` agrandit le vecteur au fil des copies : il n'est pas nécessaire de dimensionner celui-ci à l'avance. Les clés d'une `map` sont constantes dans le conteneur ; les paires copiées ici sont de simples paires modifiables. Pour ce cas particulier, les deux `lower_bound` de l'exemple précédent évitent de parcourir tout l'annuaire.

5.4.3 Algorithmes numériques

```cpp
// Programme complet
#include <iostream>
#include <numeric>
#include <vector>
int main() {
  std::vector<int> v(5);
  std::iota(v.begin(), v.end(), 1); // 1 2 3 4 5
  auto somme = std::accumulate(v.begin(), v.end(), 0);
  std::vector<int> cumul(v.size());
  std::partial_sum(v.begin(), v.end(), cumul.begin());
  std::cout << somme << ' ' << cumul.back() << '\n'; // 15 15
}
```

Le type de l'accumulateur est déduit de sa valeur initiale : `0` donne ici un `int`, tandis que `0.0` donnerait un `double`. `std::partial_sum` écrit dans une sortie assez grande pour accueillir les résultats ; contrairement à `back_inserter`, `begin()` ne fait pas grandir le conteneur.

5.5 Ranges, algorithmes et vues (C++20)

Un *range* est un objet dont on peut obtenir un début et une fin compatible : il permet souvent de passer le conteneur entier à l'algorithme. Une *vue* est un objet range conçu pour être peu coûteux à copier ou à déplacer ; elle peut représenter des éléments sélectionnés ou transformés sans les recopier immédiatement. Une vue n'est pas nécessairement non propriétaire dans tous les cas ; la vue produite à partir d'un conteneur *lvalue* dans l'exemple ci-dessous l'observe.

```cpp
// C++20 ; programme complet
#include <algorithm>
#include <iostream>
#include <ranges>
#include <vector>
int main() {
  std::vector<int> v{5, 2, 7, 2, 4};
  std::ranges::sort(v);
  auto fin = std::ranges::unique(v);
  v.erase(fin.begin(), fin.end());
  auto pairsCarres = v
    | std::views::filter([](int x) { return x % 2 == 0; })
    | std::views::transform([](int x) { return x * x; });
```

```
  for (int x : pairsCarres) std::cout << x << ' '; // 4 16
  std::cout << '\n';
}
```

`std::ranges::unique` renvoie un sous-intervalle à supprimer ; `std::unique`, lui, renvoie un itérateur. Les vues `filter` et `transform` sont évaluées au parcours. Dans cet exemple, `v` doit rester vivant et les opérations qui invalident ses itérateurs peuvent rendre l'utilisation de la vue invalide : créez-la et utilisez-la après les modifications structurelles du vecteur. Les prédicats doivent respecter les exigences sémantiques des vues et algorithmes.

Les algorithmes ranges acceptent souvent une *projection*, appliquée aux éléments avant la comparaison ou le prédicat :

```
// C++20 ; programme complet
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
struct Personne { std::string nom; int age; };
int main() {
  std::vector<Personne> v{{"Ada", 36}, {"Alan", 41}, {"Grace", 29}};
  std::ranges::sort(v, {}, &Personne::age);
  for (const auto& p : v) std::cout << p.nom << ' '; // Grace Ada Alan
}
```

Le deuxième argument `{}` choisit le comparateur par défaut ; `&Personne::age` est la projection. Les versions ranges imposent des contraintes vérifiées à la compilation, par exemple sur la catégorie de l'itérateur et la possibilité de trier ses éléments.

Un résultat de recherche peut être une référence ou un itérateur vers les données d'origine. Ne le conservez pas au-delà de leur durée de vie ni après une opération qui l'invalide. Pour certains ranges temporaires non empruntables, les algorithmes ranges retournent le type `std::ranges::dangling` afin d'empêcher l'utilisation d'un itérateur devenu invalide.

5.6 Types déduits, `decltype` et valeur maximale

`auto` déduit le type d'une variable à partir de son initialiseur : `auto` it = v.begin();. Une déclaration `auto x = *it` copie généralement une valeur, tandis que `auto& x = *it` la référence lorsque le déréférencement fournit une lvalue compatible. Si le déréférencement produit un proxy, le type de `x` peut être ce proxy ; une copie du proxy n'est pas forcément une copie de la valeur sous-jacente.

`decltype(expression)` permet d'obtenir le type d'une expression. Attention à la forme avec ou sans parenthèses pour une variable nommée :

```
// Programme complet
#include <type_traits>
int main() {
  int x = 4;
  static_assert(std::is_same<decltype(x), int>::value, "int");
  static_assert(std::is_same<decltype((x)), int&>::value, "int&");
  auto copie = x;
  auto& reference = x;
  reference = 5;
  return (copie == 4 && x == 5) ? 0 : 1;
}
```

`std::declval<T>()`, dans `<utility>`, permet de former une expression dépendant de `T` dans un contexte non évalué, sans créer d'objet. Pour le type de valeur d'un itérateur, préférez souvent `std::iterator_traits<I>::value_type`, ou `std::iter_value_t<I>` depuis C++20, à une transformation manuelle de `decltype(*it)`, notamment à cause des références et des proxies.

L'exemple suivant calcule une valeur maximale et représente explicitement l'absence de résultat pour un intervalle vide. Contrairement à une variable locale scalaire non initialisée, il n'essaie pas de lire une valeur indéterminée.

```
// C++17 ; programme complet
#include <iostream>
#include <iterator>
#include <optional>
#include <vector>
```

```cpp
template <typename I>
std::optional<typename std::iterator_traits<I>::value_type>
maximum(I debut, I fin) {
  using Valeur = typename std::iterator_traits<I>::value_type;
  if (debut == fin) return std::nullopt;
  Valeur resultat = *debut;
  for (++debut; debut != fin; ++debut)
    if (resultat < *debut) resultat = *debut;
  return resultat;
}
int main() {
  std::vector<int> v{4, 3, 18, 12, 5, 2, 13};
  auto m = maximum(v.begin(), v.end());
  if (m) std::cout << *m << '\n'; // 18
  std::vector<int> vide;
  return maximum(vide.begin(), vide.end()) ? 1 : 0;
}
```

Cette version copie la valeur initiale et affecte les maxima successifs ; elle exige notamment que ces opérations et la comparaison soient valides. Si l'on veut plutôt une position dans un intervalle multipassage, `std::max_element` renvoie un itérateur, égal à la fin si l'intervalle est vide ; il faut tester ce cas avant de déréférencer.

En résumé : Conteneurs, itérateurs et algorithmes

- Choisir le conteneur selon les opérations, leur coût et l'invalidation des itérateurs.
- Les intervalles usuels sont semi-ouverts : la borne de fin ne se déréférence pas.
- Chaque algorithme demande des capacités précises à ses itérateurs et à ses éléments.
- Les concepts C++20 distinguent notamment entrée, sortie, multipassage, accès aléatoire et contiguïté ; une sentinelle peut avoir un type distinct.
- Les algorithmes `remove` et `unique` ne réduisent pas eux-mêmes la taille d'un vecteur.
- Les ranges et vues composent des opérations ; respecter la durée de vie des données observées.
- Les conteneurs ordonnés et hachés répondent à des besoins différents ; aucun coût moyen ne garantit le pire cas.

Point de vue



Avant de réécrire un algorithme courant, cherchez s'il existe dans la bibliothèque standard et vérifiez ses préconditions. Un algorithme adapté rend souvent l'intention plus lisible ; un algorithme générique n'est pas pour autant automatiquement optimal pour tous les conteneurs. Préférez par exemple les opérations membres d'une liste ou d'un conteneur associatif lorsqu'elles exploitent sa structure.

Références techniques. Projet public du standard C++ : `https://eel.is/c++draft/`, notamment `[containers]`, `[iterator.concepts]`, `[algorithms]` et `[ranges]`. Ce projet évolue ; les versions indiquées ci-dessus permettent de distinguer les fonctionnalités présentées.

6 Programmation générique avancée

Les chapitres précédents ont présenté les patrons, les conteneurs et les algorithmes. Nous examinons ici comment exprimer les contraintes d'un algorithme, obtenir des informations sur les types et sélectionner une implémentation à la compilation. Les concepts C++20 constituent le point d'entrée principal ; les techniques historiques restent utiles pour lire ou maintenir du code plus ancien.

Un *concept*, au sens de la programmation générique, rassemble des exigences sur un ou plusieurs types et sur leurs opérations. Un type qui respecte ces exigences et leur sémantique est un *modèle* du concept. Par exemple, un algorithme de tri dépend séparément d'un intervalle et d'un comparateur, mais exige que le comparateur accepte les éléments de cet intervalle et établisse un ordre strict faible. Un concept n'est pas une classe mère : aucun héritage n'est nécessaire pour le satisfaire.

Les exemples sans indication fonctionnent en C++11 ; les versions minimales postérieures sont précisées. Les programmes complets sont autonomes.

6.1 Contraintes avec les concepts C++20

Le mot-clé `concept` nomme une contrainte. Une expression `requires` teste la validité d'opérations et, si besoin, leurs types de résultat ou l'absence d'exception. Ces vérifications ne prouvent pas les propriétés mathématiques attendues : une comparaison qui compile n'est pas nécessairement transitive.

```
// C++20 ; programme complet
#include <concepts>
#include <cstddef>
#include <string>
#include <vector>

template <typename T>
concept Mesurable = requires(const T& x) {
  { x.size() } -> std::convertible_to<std::size_t>;
};

template <Mesurable T>
std::size_t taille(const T& x) { return x.size(); }

int main() {
  std::vector<int> v{1, 2, 3};
  std::string s = "chat";
  return taille(v) == 3 && taille(s) == 4 ? 0 : 1;
  // taille(42); // invalide : l'expression x.size() n'existe pas
}
```

Le compilateur vérifie ici l'expression `x.size()` pour un objet constant et la convertibilité de son résultat en `std::size_t`. Il ne vérifie pas que le résultat correspond effectivement au nombre d'éléments. Les paramètres écrits entre parenthèses après `requires` servent à formuler les exigences ; aucun objet n'est construit par cette expression.

Une même contrainte peut être placée à plusieurs endroits :

```
// C++20 ; fragments equivalents utilisant Mesurable ci-dessus
template <typename T> requires Mesurable<T>
std::size_t taille1(const T& x) { return x.size(); }

template <typename T>
std::size_t taille2(const T& x) requires Mesurable<T> {
  return x.size();
}

std::size_t taille3(const Mesurable auto& x) { return x.size(); }
```

Attention : deux paramètres écrits séparément sous la forme `Mesurable auto` déduisent deux types indépendants. Pour imposer le même type aux deux arguments, utilisez un même paramètre nommé `T` ou une contrainte `std::same_as` explicite.

6.1.1 Réutilisation et choix entre surcharges

Les concepts peuvent être composés. En réutilisant le concept nommé dans une contrainte plus forte, on permet au compilateur de reconnaître la relation entre les surcharges :

```
// C++20 ; programme complet
#include <concepts>
#include <cstddef>
#include <list>
#include <vector>
```

```cpp
template <typename T>
concept Mesurable = requires(const T& x) {
  { x.size() } -> std::convertible_to<std::size_t>;
};
template <typename T>
concept MesurableIndexable = Mesurable<T> && requires(const T& x) {
  x[std::size_t{0}];
};

template <Mesurable T>
int categorie(const T&) { return 1; }
template <MesurableIndexable T>
int categorie(const T&) { return 2; }

int main() {
  std::list<int> l{1, 2};
  std::vector<int> v{1, 2};
  return categorie(l) == 1 && categorie(v) == 2 ? 0 : 1;
}
```

La surcharge la plus contrainte est retenue lorsque la relation de *subsomption* est établie et que les autres règles de résolution des surcharges le permettent. Le compilateur ne démontre pas toutes les implications logiques entre expressions arbitraires : répéter séparément deux expressions `requires` semblables ne crée pas toujours des contraintes atomiques identiques. Nommer et réutiliser les concepts aide à rendre cet ordre explicite. L'expression `x[0]` dans le concept n'est pas évaluée sur un conteneur vide ; elle teste sa validité syntaxique.

6.1.2 Quelques concepts standard

Dans `<concepts>`, les familles suivantes sont particulièrement utiles :

- Types et conversions : `same_as`, `derived_from`, `convertible_to`, `common_with` et `common_reference_with`. `derived_from` demande une base publique et non ambiguë, alors que `is_base_of` peut aussi détecter une base privée ou ambiguë.
- Types numériques : `integral`, `signed_integral`, `unsigned_integral` et `floating_point`.
- Construction et affectation : `constructible_from`, `default_initializable`, `copy_constructible`, `move_constructible`, `assignable_from`, `swappable` et `swappable_with`.
- Modèles de valeur : `movable`, `copyable`, `semiregular` et `regular`. Ce dernier combine notamment une sémantique de valeur et une comparaison d'égalité.
- Relations et appels : `equality_comparable`, `totally_ordered`, `invocable`, `predicate`, `relation`, `equivalence_relation` et `strict_weak_order`. Les relations mathématiques qu'ils nomment doivent être respectées par le programmeur.

`regular_invocable` a des exigences sémantiques plus fortes que `invocable`, notamment la stabilité des résultats dans les conditions requises, sans que le compilateur puisse la déduire d'un simple test syntaxique. Les concepts d'itérateurs sont déclarés dans `<iterator>`, les concepts de ranges dans `<ranges>` et ceux de comparaison à trois voies dans `<compare>` ; voir aussi la partie 5.

6.2 Propriétés et transformations de types

L'en-tête `<type_traits>` fournit des patrons décrivant des propriétés et des relations entre types : leurs résultats sont disponibles à la compilation. Quelques familles utiles :

- Catégories : `is_void`, `is_integral`, `is_floating_point`, `is_array`, `is_enum`, `is_class`, `is_function`, `is_pointer` et `is_reference`.
- Propriétés : `is_const`, `is_trivially_copyable`, `is_polymorphic`, `is_abstract`, `is_signed` et `is_unsigned`.
- Opérations et relations : `is_constructible`, `is_copy_constructible`, `is_assignable`, `has_virtual_destructor`, `is_same`, `is_base_of`, `is_convertible` et `is_invocable` (C++17). `is_swappable` est également disponible depuis C++17.

`is_pointer` ne reconnaît pas les pointeurs vers membres, qui forment une catégorie distincte testée par `is_member_pointer`. La référence `const int&` n'est pas elle-même qualifiée `const` ; pour tester la qualification du type référencé, on retire d'abord la référence.

Les traits de propriété exposent généralement `::value`. Depuis C++17, les raccourcis `_v` donnent directement cette valeur, par exemple `std::is_integral_v<T>`. Les transformations fournissent un type par `::type` ou, depuis C++14, par un alias `_t` :

```
// Programme complet
#include <type_traits>
int main() {
  static_assert(std::is_integral<int>::value, "int est entier");
  static_assert(!std::is_integral<double>::value, "double n'est pas entier");
  static_assert(!std::is_const<const int&>::value, "reference non const");
  typedef std::remove_reference<const int&>::type SansReference;
  static_assert(std::is_same<SansReference, const int>::value, "const int");
  typedef std::remove_cv<SansReference>::type Valeur;
  static_assert(std::is_same<Valeur, int>::value, "int");
}
```

Parmi les transformations figurent `remove_reference`, `remove_cv`, `add_pointer`, `remove_extent`, `make_signed`, `decay` et `conditional`. C++20 ajoute `remove_cvref`, qui retire référence et qualifications de premier niveau ; contrairement à `decay`, il ne convertit pas les tableaux ou les fonctions en pointeurs. `std::conditional<B, T, U>::type` choisit un type selon le booléen `B`.

Le choix « passer par valeur si `sizeof(T)` est petit » est une heuristique, pas une règle générale : coût de copie, ABI et représentation comptent aussi. De plus, un paramètre écrit comme `typename Traits<T>::type` peut empêcher la déduction de `T` à l'appel. Une interface devrait documenter ses choix et les mesurer au besoin.

6.3 Types associés et classes traits

Un type associé décrit une relation utile à une abstraction. Pour un itérateur lisible `I`, le type de valeur et le type de l'expression `*i` peuvent différer. `std::iterator_traits<I>::value_type` fonctionne également pour les pointeurs, qui ne possèdent aucun type membre. Depuis C++20, `std::iter_value_t<I>` et `std::iter_reference_t<I>` expriment directement ces deux notions ; le second peut être un proxy.

```
// C++20 ; programme complet
#include <iterator>
#include <type_traits>
#include <vector>
int main() {
  using I = std::vector<int>::const_iterator;
  static_assert(std::is_same_v<std::iter_value_t<I>, int>);
  static_assert(std::is_same_v<std::iter_reference_t<I>, const int&>);
  static_assert(std::is_same_v<std::iter_value_t<int*>, int>);
}
```

Les classes traits associent ainsi des types ou constantes à un type, et peuvent être spécialisées lorsque les règles de la bibliothèque ou de l'interface l'autorisent. Il ne faut pas ajouter arbitrairement des spécialisations dans `std`. Pour ses propres types, un trait défini par une bibliothèque utilisatrice peut exposer une valeur par défaut et des spécialisations ciblées.

6.4 Sélection à la compilation : `if constexpr` et appelables

Une condition `if constexpr` dépendant des paramètres d'un template permet, lors de l'instanciation, d'écarter une branche dont les opérations ne conviennent pas au type concerné. Elle existe depuis C++17. Un `if` ordinaire ne suffit pas : les deux branches doivent être valides après instanciation. Même avec `if constexpr`, une erreur indépendante des paramètres du template n'est pas automatiquement rendue légale.

```
// C++17 ; programme complet
#include <iostream>
#include <string>
#include <type_traits>
template <typename T>
void affiche(const T& x) {
  if constexpr (std::is_integral<T>::value)
    std::cout << "entier : " << x + 1 << '\n';
  else
```

```
    std::cout << "autre : " << x << '\n';
}
int main() {
  affiche(4);                       // entier : 5
  affiche(std::string("bonjour")); // autre : bonjour
}
```

La condition est évaluée à la compilation pour chaque spécialisation ; une fonction `constexpr` appelée avec un argument d'exécution peut, elle, s'exécuter à l'exécution. Ce sont deux mécanismes distincts.

`std::invoke`, depuis C++17, appelle uniformément fonctions, foncteurs, lambdas et pointeurs de membres. Avec `std::invocable`, C++20 permet de contraindre un algorithme sur l'expression appelée et le type effectivement renvoyé par le déréférencement :

```
// C++20 ; programme complet
#include <concepts>
#include <functional>
#include <iterator>
#include <vector>
template <std::input_iterator I, std::sentinel_for<I> S, typename F>
  requires std::invocable<F&, std::iter_reference_t<I>>
void applique(I debut, S fin, F f) {
  for (; debut != fin; ++debut) std::invoke(f, *debut);
}
int main() {
  std::vector<int> v{1, 2, 3};
  applique(v.begin(), v.end(), [](int& x) { x *= 2; });
  return v == std::vector<int>{2, 4, 6} ? 0 : 1;
}
```

La fonction prend ici l'appelable par valeur : un foncteur à état est donc copié ou déplacé. `std::ref` permet explicitement de transmettre une référence. La validité de l'appel ne prouve pas que l'opération respecte des exigences sémantiques plus fortes, comme celles d'un prédicat régulier.

6.5 SFINAE et détection : lecture du code antérieur à C++20

SFINAE signifie *Substitution Failure Is Not An Error*. Lorsqu'une substitution échoue dans le contexte immédiat de la signature ou de certains paramètres d'un candidat template, ce candidat peut être écarté de la résolution des surcharges au lieu de provoquer immédiatement une erreur. S'il ne reste aucun candidat approprié, ou si le choix est ambigu, l'appel reste invalide. Les erreurs dans un corps de fonction instancié et les effets secondaires de l'instanciation ne sont pas tous couverts par SFINAE.

Avant les concepts, `std::enable_if` servait souvent à exclure une surcharge, et `std::void_t` (C++17) à détecter une expression. Voici une détection de l'appel `a.swap(b)` ; elle vérifie l'opération, plutôt que la seule existence d'un membre nommé `swap` :

```
// C++17 ; programme complet
#include <type_traits>
#include <utility>
#include <vector>

template <typename T, typename = void>
struct possede_swap : std::false_type {};

template <typename T>
struct possede_swap<T, std::void_t<
  decltype(std::declval<T&>().swap(std::declval<T&>()))
>> : std::true_type {};

struct MauvaisSwap { void swap() {} };
int main() {
  static_assert(possede_swap<std::vector<int>>::value);
  static_assert(!possede_swap<int>::value);
  static_assert(!possede_swap<MauvaisSwap>::value);
}
```

`std::declval` n'est utilisé que dans un contexte non évalué. Le trait vérifie que l'expression est bien formée, pas qu'elle échange réellement les valeurs ni qu'elle soit rapide. Tester `&T::swap` seul est moins précis : il peut accepter une signature inadaptée ou échouer sur des surcharges.

En C++20, le même test d'expression s'écrit plus directement :

```
// C++20 ; fragment
template <typename T>
concept PossedeSwapMembre = requires(T& a, T& b) {
  a.swap(b);
};
```

Il ne faut pas le confondre avec `std::swappable`, qui concerne l'échange accessible par les mécanismes appropriés, y compris une fonction libre trouvée par recherche dépendant des arguments (ADL). Dans un algorithme classique, `using std::swap; swap(a, b);` permet de considérer la surcharge standard et les surcharges trouvées par ADL ; en C++20, `std::ranges::swap(a, b)` est un autre point de personnalisation adapté. On ne peut pas supposer qu'une méthode membre est toujours le seul moyen correct ou le plus rapide d'échanger.

6.6 Dispatch par marques et contraintes sur les itérateurs

Les catégories classiques d'itérateurs sont représentées par des marques (*tags*) dans `<iterator>`. La surcharge peut choisir une implémentation en fonction de la marque renvoyée par `iterator_traits`. Voici une version pédagogique de la distance pour des itérateurs classiques multipassage :

```
// Programme complet
#include <iterator>
#include <list>
#include <vector>

template <typename I>
typename std::iterator_traits<I>::difference_type
mesure(I debut, I fin, std::input_iterator_tag) {
  typename std::iterator_traits<I>::difference_type n = 0;
  for (; debut != fin; ++debut) ++n;
  return n;
}
template <typename I>
typename std::iterator_traits<I>::difference_type
mesure(I debut, I fin, std::random_access_iterator_tag) {
  return fin - debut;
}
template <typename I>
typename std::iterator_traits<I>::difference_type
distance_pedagogique(I debut, I fin) {
  typedef typename std::iterator_traits<I>::iterator_category Categorie;
  return mesure(debut, fin, Categorie{});
}
int main() {
  std::vector<int> v{1, 2, 3, 4};
  std::list<int> l{1, 2, 3};
  return distance_pedagogique(v.begin(), v.end()) == 4
      && distance_pedagogique(l.begin(), l.end()) == 3 ? 0 : 1;
}
```

Pour un itérateur aléatoire, la soustraction est constante ; pour la liste, la boucle est linéaire. Les deux bornes doivent former un intervalle valide, et la distance doit être représentable. Cette fonction n'est qu'une illustration : `std::distance` fournit le service standard. Les marques historiques de lecture ont une hiérarchie d'héritage ; `output_iterator_tag` ne s'insère pas comme une classe de base de `forward_iterator_tag`. Les concepts C++20 formalisent autrement les capacités et peuvent distinguer une sentinelle de l'itérateur.

Lorsque l'on choisit une surcharge selon des catégories d'itérateurs, les exigences de l'algorithme ne se résument pas à la catégorie : un tri exige aussi un ordre approprié, la possibilité de déplacer et d'échanger les éléments, et une sortie modifiable. Par exemple, un itérateur de `set` est bidirectionnel mais ne permet pas de réordonner ses clés. En C++20, `std::sortable<I>` complète les contraintes de catégorie pour les algorithmes de tri concernés.

6.7 Méta-programmation old-school

Le compilateur C++ peut compiler des expressions très complexes, et en réalité, effectuer des calculs comme un programme en utilisant les mécanismes liés aux classes patrons et à leur spécialisation. On parle de *méta-programmation*. On peut par exemple précalculer certaines expressions pour qu'à l'exécution le processeur n'est pas à les recalculer. Un exemple simple est donné ci-dessous, où on se débrouille pour faire calculer par le compilateur la factorielle de n'importe quel entier (pas trop grand).

```
#include <iostream>
using namespace std;

template <int N>
struct Factorielle {
  static const int value = N * Factorielle< N-1 >::value;
};
template <>
struct Factorielle<1> {
  static const int value = 1;
};

int main()
{
  cout << "1! = " << Factorielle<1>::value << endl;
  cout << "2! = " << Factorielle<2>::value << endl;
  cout << "3! = " << Factorielle<3>::value << endl;
  cout << "4! = " << Factorielle<4>::value << endl;
  cout << "5! = " << Factorielle<5>::value << endl;
}
```

6.8 Calculs et expressions constantes

La métaprogrammation peut calculer des valeurs ou des types pendant la compilation. Les spécialisations de classes permettaient déjà de calculer une factorielle récursivement ; les fonctions `constexpr` expriment souvent le calcul plus simplement. Le résultat est calculé à la compilation lorsqu'un contexte constant l'exige ; un appel avec des valeurs d'exécution n'est pas nécessairement évalué à la compilation.

```
// Programme complet
#include <iostream>
constexpr unsigned long long factorielle(unsigned n) {
  return n <= 1 ? 1ULL : n * factorielle(n - 1);
}
int main() {
  static_assert(factorielle(0) == 1, "0! = 1");
  static_assert(factorielle(5) == 120, "5! = 120");
  unsigned n = 0;
  if (std::cin >> n && n <= 20) std::cout << factorielle(n) << '\n';
}
```

La valeur doit tenir dans le type choisi ; les grands arguments débordent ici. C++14 autorise notamment des corps `constexpr` plus riches avec boucles et variables locales. C++20 ajoute `consteval` pour une fonction immédiate, dont les appels soumis aux règles d'évaluation immédiate doivent produire une expression constante, ainsi que `constinit` pour imposer l'initialisation statique d'une variable appropriée. `constinit` ne rend pas cette variable constante.

```
// C++20 ; programme complet
consteval int carre(int x) { return x * x; }
constinit int compteur = carre(4); // valeur initiale 16, modifiable
int main() {
  ++compteur;
  return compteur == 17 ? 0 : 1;
}
```

6.9 Packs de paramètres et transmission parfaite

Les templates variadiques et la récursion sur un pack ont été introduits dans la partie 4. Depuis C++17, une expression de repli peut par exemple appliquer une opération à tous les arguments :

```
// C++17 ; programme complet
#include <iostream>
template <typename... T>
void affiche_tout(const T&... valeurs) {
  ((std::cout << valeurs << ' '), ...);
  std::cout << '\n';
}
int main() { affiche_tout(3, 2.5, "chat"); }
```

Le repli sur la virgule impose ici un parcours des opérations de gauche à droite ; le pack vide est accepté et produit seulement un saut de ligne.

Une *référence de transmission* est de la forme `T&&` lorsque `T` est un paramètre de type déduit dans les conditions appropriées. À l'appel, elle accepte lvalues et rvalues ; `std::forward<T>` restitue la catégorie de l'argument dans une transmission :

```
// Programme complet
#include <iostream>
#include <utility>
void traite(int&)  { std::cout << "lvalue\n"; }
void traite(int&&) { std::cout << "rvalue\n"; }
template <typename T>
void transmet(T&& x) {
  traite(std::forward<T>(x));
}
int main() {
  int n = 4;
  transmet(n); // T = int&, puis int& && devient int&
  transmet(4); // T = int,  puis int&&
}
```

Une expression nommant `x` est une lvalue, même lorsque son type déclaré est une référence de rvalue. `std::move` autorise un déplacement sans préserver cette distinction ; il ne remplace donc pas systématiquement `std::forward`. La forme `const T&&` n'est pas une référence de transmission de ce type.

En résumé : Programmation générique avancée

- Concevoir un algorithme générique demande d'identifier ses opérations et leurs propriétés sémantiques.
- Les concepts C++20 expriment des contraintes et améliorent la sélection des surcharges et les diagnostics.
- Traits et types associés donnent accès aux propriétés des types et aux types produits par les abstractions.
- `if constexpr`, les contraintes et le dispatch par tags sélectionnent du code à la compilation.
- SFINAE explique des interfaces historiques mais n'efface pas toute erreur.
- `constexpr` permet, sans toujours l'imposer, un calcul à la compilation ; la transmission parfaite préserve la catégorie de l'argument.

Point de vue



Privilégiez une contrainte qui exprime le besoin réel de l'algorithme. Une hiérarchie de types ou une méthode particulière n'est pas nécessairement l'abstraction recherchée. Les concepts rendent l'interface plus claire, mais les propriétés sémantiques, les préconditions et la complexité doivent encore être documentées et testées.

Références techniques. Projet public du standard C++ : `https://eel.is/c++draft/`, en particulier `[temp.deduct]`, `[temp.constr]`, `[iterator.traits]` et `[dcl.constexpr]`.

7 Expressions lambda, exceptions et autres outils du langage

Cette partie présente les expressions lambda et quelques constructions utiles pour écrire un code plus concis. Les concepts, les traits, SFINAE et les calculs à la compilation sont regroupés dans la partie 6. Les exemples sans indication fonctionnent en C++11 ; les versions minimales postérieures sont précisées.

7.1 Expressions lambda et objets fonction

Une *lambda* crée un objet fonction, appelé *fermeture* (*closure*), dont le type est propre à cette expression. Son opérateur `()` exécute le corps de la lambda. La forme courante est :

```
[captures](parametres) { /* corps */ }
```

Les captures rendent accessibles des variables du contexte. Comme pour tout appelable, on peut passer la fermeture à un algorithme. L'exemple suivant retire d'un vecteur les éléments strictement inférieurs à un seuil :

```
// Programme complet
#include <algorithm>
#include <iostream>
#include <vector>
int main() {
  std::vector<int> v{12, 6, 9, 14, 15, 19, 5};
  const int seuil = 10;
  auto fin = std::remove_if(v.begin(), v.end(),
                            [seuil](int x) { return x < seuil; });
  v.erase(fin, v.end());
  for (int x : v) std::cout << x << ' '; // 12 14 15 19
  std::cout << '\n';
}
```

L'algorithme `remove_if` compacte les éléments conservés ; `erase` réduit effectivement le vecteur. Le corps de la lambda peut accéder à la copie de `seuil` créée lors de sa construction.

7.2 Captures par valeur et par référence

Les captures les plus fréquentes sont les suivantes :

— `[]` : aucune capture.
— `[a, &b]` : capture de `a` par valeur et de `b` par référence.
— `[=]` : capture par valeur des entités locales éligibles utilisées implicitement par le corps.
— `[&]` : capture par référence des entités locales éligibles utilisées implicitement par le corps.
— `[this]` : capture du pointeur `this`, pas une copie de l'objet désigné.

Les variables de durée statique accessibles dans la portée peuvent être utilisées sans capture ; les captures par défaut ne copient donc pas « toutes les variables » du contexte.

```
// Programme complet
#include <iostream>
int main() {
  int seuil = 10;
  int compteur = 0;
  auto teste = [seuil, &compteur](int x) {
    if (x < seuil) { ++compteur; return true; }
    return false;
  };
  seuil = 20; // ne modifie pas la copie deja capturee
  teste(8);
  teste(15);
  std::cout << compteur << '\n'; // 1
}
```

Une capture par référence n'allonge pas la durée de vie de l'objet référencé : la lambda ne doit pas être appelée après sa destruction. Le même problème se pose avec `this` si la fermeture survit à l'objet.

À partir de C++17, `[*this]` capture une copie de l'objet courant ; la copie éventuelle de membres pointeurs ne prolonge pas, à elle seule, la vie de leurs cibles. La capture implicite de `this` par `[=]` est dépréciée depuis C++20 : lorsque c'est pertinent, écrivez la capture explicitement.

Par défaut, l'opérateur d'appel d'une lambda ordinaire est constant : les valeurs capturées par copie ne peuvent pas être modifiées directement dans son corps. `mutable` rend cet opérateur non constant et permet de modifier les copies, sans modifier les variables originales :

```
// Programme complet
#include <iostream>
int main() {
  int n = 0;
  auto prochain = [n]() mutable { return ++n; };
  int premier = prochain();
  int deuxieme = prochain();
  std::cout << premier << ' ' << deuxieme << ' ' << n << '\n'; // 1 2 0
}
```

La fermeture possède sa propre copie de `n`, dont la valeur persiste entre les appels à `prochain`.

Depuis C++14, une capture initialisée peut déplacer une ressource dans la fermeture, sans conserver de référence vers une variable locale :

```
// C++14 ; programme complet
#include <memory>
#include <utility>
int main() {
  auto p = std::make_unique<int>(7);
  auto lire = [valeur = std::move(p)] { return *valeur; };
  // p est vide ; lire possede maintenant l'entier.
  // auto copie = lire; // invalide : unique_ptr n'est pas copiable
  return lire() == 7 ? 0 : 1;
}
```

La durée de vie de l'entier est attachée à celle de la fermeture propriétaire. Une capture initialisée par référence, au contraire, conserve les contraintes de durée de vie de toute référence.

7.3 Type de retour et paramètres génériques

Lorsque c'est possible, le compilateur déduit le type de retour à partir des instructions `return`. On peut l'indiquer explicitement avec `-> Type`, notamment pour documenter l'interface ou imposer une conversion. Une lambda sans capture peut être convertie en pointeur de fonction compatible ; une lambda avec capture doit conserver son état dans un objet fonction.

```
// Programme complet
#include <iostream>
int main() {
  auto moyenne = [](double a, double b) -> double {
    return (a + b) / 2.0;
  };
  double (*f)(double, double) = moyenne; // aucune capture
  std::cout << f(2.0, 5.0) << '\n'; // 3.5
}
```

Depuis C++14, un paramètre `auto` définit une lambda générique :

```
// C++14 ; programme complet
#include <string>
int main() {
  auto addition = [](const auto& a, const auto& b) { return a + b; };
  auto n = addition(2, 3);
  auto s = addition(std::string("bon"), std::string("jour"));
  return n == 5 && s == "bonjour" ? 0 : 1;
}
```

Les deux paramètres `auto` sont déduits indépendamment. Depuis C++20, une liste explicite de paramètres template permet de partager un type ou de le contraindre :

```
// C++20 ; programme complet
#include <concepts>
int main() {
  auto somme = []<std::integral T>(T a, T b) { return a + b; };
  return somme(2, 3) == 5 ? 0 : 1;
```

```
  // somme(2, 3.5); // deduction incompatible pour le meme T
}
```

7.4 Conserver et transmettre des appelables

Le type d'une lambda n'a pas de nom accessible directement dans le code source ; `auto` conserve néanmoins son type précis et toutes les informations nécessaires au compilateur. Deux expressions lambda distinctes ont des types distincts, même si leurs corps se ressemblent.

Un paramètre template conserve également le type concret de l'appelable. Dans `<functional>`, `std::function` permet de stocker sous un type commun des appelables différents compatibles avec une même signature :

```
// Programme complet
#include <cmath>
#include <functional>
#include <iostream>
struct Carre {
  double operator()(double x) const { return x * x; }
};
int main() {
  std::function<double(double)> f = Carre{};
  std::cout << f(2.0) << '\n'; // 4
  f = [](double x) { return std::sin(x); };
  std::cout << f(0.0) << '\n'; // 0
  f = nullptr; // fonction vide
  if (!f) std::cout << "aucune fonction\n";
}
```

L'appel d'un `std::function` vide lève `std::bad_function_call`. L'effacement du type peut entraîner un appel indirect et, selon le type et l'implémentation, une allocation ; il faut mesurer si ce coût importe. `std::function` exige une cible copiable : la lambda propriétaire d'un `unique_ptr` présentée plus haut ne convient pas directement. C++23 introduit `std::move_only_function` pour des appelables seulement déplaçables ; sa disponibilité dépend de la bibliothèque standard.

7.5 Liaisons structurées et initialisation dans une condition

Depuis C++17, une *liaison structurée* permet de nommer les composantes d'une paire, d'un tuple ou de certains objets :

```
// C++17 ; programme complet
#include <iostream>
#include <map>
#include <string>
int main() {
  std::map<std::string, int> notes{{"Ada", 18}, {"Alan", 16}};
  for (const auto& [nom, note] : notes)
    std::cout << nom << " : " << note << '\n';
  auto [it, insere] = notes.emplace("Grace", 19);
  if (insere) std::cout << it->first << " ajoutee\n";
}
```

Le `&` évite ici de copier chaque paire lors du parcours ; les clés d'une `map` restent non modifiables. Sans référence, la liaison structurée crée généralement un objet intermédiaire par copie ou déplacement. La déclaration avec initialisation d'un `if` permet aussi de limiter la portée d'une variable :

```
// C++17 ; programme complet
#include <map>
#include <string>
int main() {
  std::map<std::string, int> notes{{"Ada", 18}};
  if (auto it = notes.find("Ada"); it != notes.end())
    return it->second == 18 ? 0 : 1;
  return 1; // it n'est plus visible ici
}
```

Une syntaxe analogue existe pour `switch`. L'objet initialisé dans la condition reste vivant dans les branches associées, puis est détruit à la sortie de l'instruction.

7.6 Traitement des exceptions

Une exception permet de signaler une erreur qu'une fonction ne peut pas ou ne souhaite pas traiter elle-même. L'expression `throw` lance une exception ; le programme recherche un bloc `catch` compatible associé à un bloc `try`. Si la fonction courante ne la traite pas, l'exception se propage à son appelant. En C++, le mot-clé est `throw`, et non `throws`.

```
// Programme complet
#include <exception>
#include <iostream>
#include <stdexcept>
#include <vector>

double moyenne(const std::vector<double>& valeurs) {
  if (valeurs.empty())
    throw std::invalid_argument("moyenne d'un tableau vide");
  double somme = 0.0;
  for (double x : valeurs) somme += x;
  return somme / valeurs.size();
}

int main() {
  try {
    std::vector<double> valeurs{2.0, 4.0, 6.0};
    std::cout << "Moyenne : " << moyenne(valeurs) << '\n';
    std::vector<double> vide;
    double resultat = moyenne(vide); // lance invalid_argument
    std::cout << resultat << '\n';  // non execute
  } catch (const std::invalid_argument& erreur) {
    std::cerr << "Argument invalide : " << erreur.what() << '\n';
  } catch (const std::exception& erreur) {
    std::cerr << "Autre erreur : " << erreur.what() << '\n';
  }
}
```

`std::invalid_argument`, déclaré dans `<stdexcept>`, dérive indirectement de `std::exception`. Sa méthode `what()` fournit un message descriptif. Les gestionnaires sont examinés dans l'ordre : un `catch` pour une classe dérivée doit donc précéder celui de sa classe de base. On capture généralement une exception de classe par référence constante, afin d'éviter une copie et de préserver son type dynamique. `catch (...)` intercepte tous les types d'exceptions ; dans un `catch`, `throw;` relance l'exception en cours sans en changer le type.

Pendant la propagation vers un gestionnaire compatible, les objets locaux déjà construits dans les blocs quittés sont détruits : c'est le déroulement de pile (*stack unwinding*). Les objets propriétaires tels que `std::vector` et `std::unique_ptr` libèrent ainsi leurs ressources, conformément au principe RAII présenté dans la partie 3. Si aucune exception n'est traitée, `std::terminate` est appelé ; il ne faut pas compter sur un déroulement complet de la pile dans ce cas.

7.6.1 La spécification `noexcept`

Une fonction déclarée `noexcept` s'engage à ne pas laisser une exception sortir de son corps. Elle peut toutefois appeler une opération susceptible de lancer, à condition de traiter l'exception à l'intérieur. Si une exception s'échappe malgré tout de la fonction, le programme appelle `std::terminate` : l'appelant ne peut pas la récupérer.

```
// Fragment ; #include <vector>
bool est_vide(const std::vector<double>& v) noexcept {
  return v.empty();
}
static_assert(noexcept(est_vide(std::vector<double>{})),
              "L'appel est declare sans exception");
```

`noexcept` seul équivaut à `noexcept(true)` ; la forme `noexcept(condition)` permet une spécification conditionnelle. L'opérateur `noexcept(expression)`, comme dans l'exemple, teste à la compilation si l'expression est considérée comme ne pouvant pas lancer ; il n'exécute pas cette expression. Cette propriété concerne les déclarations et les opérations impliquées, et ne prouve pas que leur implémentation respecte son engagement.

Les destructeurs sont généralement non propagateurs d'exceptions ; une exception qui s'échappe d'un destructeur pendant le déroulement de pile appelle `std::terminate`. La spécification `noexcept` des opérations de déplacement peut aussi influencer le choix d'implémentation de certains conteneurs, comme `std::vector`, lorsqu'ils réallouent leurs éléments. Il faut donc déclarer une fonction `noexcept` seulement si cet engagement est justifié. Les anciennes spécifications dynamiques comme `throw(T)` sont supprimées depuis C++17 ; `throw()` est une ancienne forme non propagatrice, conservée pour compatibilité et dépréciée depuis C++11. Pour le code moderne, utilisez `noexcept`.

7.7 Quelques attributs et variables en ligne

Les attributs standard donnent des indications ou imposent certaines règles :

— `[[nodiscard]]` (C++17) demande un avertissement si un résultat est ignoré dans les cas prévus par le langage ; il ne remplace pas un traitement d'erreur.

— `[[maybe_unused]]` (C++17) indique qu'une entité peut rester inutilisée sans avertissement correspondant.

— `[[fallthrough]]` (C++17) documente le passage intentionnel d'un cas de `switch` au suivant.

```cpp
#include <iostream>

// 1. [[noreturn]] : indique que la fonction ne retourne jamais (ex: elle lève une exception ou coupe le programme)
[[noreturn]] void arreterProgramme(const std::string& message) {
    std::cerr << "Erreur fatale : " << message << '\n';
    std::exit(EXIT_FAILURE);
}

// 2. [[deprecated]] : indique qu'une fonction est obsolète (le compilateur émettra un avertissement si elle est utilisée)
[[deprecated("Utilisez nouvelleFonction() à la place.")]]
void ancienneFonction() {
    std::cout << "C'est l'ancienne fonction.\n";
}

// 3. [[nodiscard]] : demande au compilateur de générer un avertissement si la valeur de retour est ignorée
[[nodiscard]] int calculerSomme(int a, int b) {
    return a + b;
}

void analyserCode(int code, [[maybe_unused]] int optionCachee) {
    // 4. [[maybe_unused]] : indique au compilateur de ne pas avertir si 'optionCachee' n'est pas utilisée
    switch (code) {
        case 1:
            std::cout << "Code 1\n";
            [[fallthrough]]; // 5. [[fallthrough]] : indique explicitement qu'on veut traverser dans le case suivant sans break
        case 2:
            std::cout << "Code 2 ou 1\n";
            break;
        default:
            std::cout << "Autre code\n";
            break;
    }
}

int main() {
    // Appel d'une fonction nodiscard : le compilateur peut avertir si on ne stocke ou n'utilise pas le résultat
    int resultat = calculerSomme(5, 10);
    std::cout << "Somme : " << resultat << '\n';
    // Appel d'une fonction obsolète
    ancienneFonction();
    analyserCode(1, 42);
    return 0;
}
```

Depuis C++17, une variable `inline` définie de façon identique dans un en-tête peut être définie dans plusieurs unités de traduction sous les règles du langage, tout en désignant la même entité lorsqu'elle a la liaison appropriée. Pour une donnée membre statique mutable, on peut ainsi écrire :

```
// C++17 ; programme complet
struct Compteur {
  inline static int nombre{0};
  Compteur() { ++nombre; }
  ~Compteur() { --nombre; }
  Compteur(const Compteur&) = delete;
  Compteur& operator=(const Compteur&) = delete;
};
int main() {
  Compteur a;
  return Compteur::nombre == 1 ? 0 : 1;
}
```

Le mot-clé `inline` concerne ici les règles de définition, et ne promet pas une optimisation particulière. La copie est interdite pour que le compteur reste correct dans cet exemple minimal ; voir la partie 2 pour gérer également la copie et le déplacement.

En résumé : Expressions lambda, exceptions et outils du langage

- Une lambda crée une fermeture dont les captures constituent l'état.
- Une capture par référence ou par `this` ne prolonge pas la vie de l'objet référencé.
- `mutable` autorise la modification des copies capturées ; une capture initialisée peut transférer une ressource.
- Une lambda générique déduit les types de ses paramètres ; C++20 permet aussi de les contraindre.
- `std::function` efface le type concret d'une cible copiable ; un template conserve ce type.
- `throw`, `try` et `catch` permettent de signaler, propager et traiter une exception.
- `noexcept` indique qu'une fonction ne laisse pas échapper d'exception.
- Les liaisons structurées et les initialisations dans `if` simplifient les traitements de paires et de recherches.

Point de vue



Choisissez les captures en fonction de la durée de vie de la fermeture. Une capture explicite, telle que `[seuil, &compteur]`, documente ses dépendances et rend plus faciles à repérer les références susceptibles de dépasser leur portée. Pour les traitements locaux, la simplicité des lambdas permet souvent d'utiliser directement les algorithmes standard.

Référence technique. Projet public du standard C++ : `https://eel.is/c++draft/`, notamment `[expr.prim.lambda]`, `[function.objects]` et `[except]`.